CS 300 – Chapter 3 Lectures

Introduction to Data Structures

Jose de Lima

April 23, 2026

Key idea Chapter map

Where this sits in CS 300. Chapter 3 is the longest chapter in the course and the densest conceptually. It does four things in sequence: (1) grounds data representation (bits, ASCII) so the later structures have a floor; (2) introduces **data structures** as named shapes for storing data; (3) introduces **abstract data types** as the interface-level view; (4) builds the vocabulary of **algorithm runtime**: linear vs. binary search, Big-O, and the sorting zoo. Every later chapter in this course assumes you can read "$O(n \log n)$" and understand *why*.

What you add to your toolkit here:

- **A map of the course.** The seven structures (array, linked list, tree, BST, heap, hash table, graph) are named and contrasted before you build any of them.
- **ADT thinking.** Stacks, queues, sets, maps – the contract (interface) is what users care about; the data structure (implementation) is what you choose.
- **Big-O fluency.** You can read, write, and justify complexity claims: linear search is $O(n)$ because it touches every element; binary search is $O(\log n)$ because each step halves the remaining range.
- **Seven sorting algorithms** with their tradeoffs. Insertion, selection, shell (incremental); quicksort, merge sort (divide-and-conquer); radix (non-comparison). Knowing *which* to reach for is a high-frequency exam question.

Mastery checklist – you should be able to, without references:

1. Translate between bits, hex, and an ASCII character (e.g., `0x41` $\rightarrow$ `'A'`).
2. State the interface of each ADT (List, Stack, Queue, Set, Map, Priority Queue) and give one concrete data structure that implements it.
3. Explain linear vs. binary search with the runtime for each on the worst case and the preconditions.
4. State the Big-O definition from memory ("$f(n) = O(g(n))$ iff there exist constants c and $n_0 \ldots$") and apply it to a small example.
5. Order the growth hierarchy: $1, \log n, n, n \log n, n^2, 2^n$.
6. For each sort (insertion, selection, shell, quicksort, merge, radix), give best / average / worst / stable? / in-place? *and* the one sentence that explains the runtime.
7. Pick the right sort for a scenario: "small, nearly sorted," "huge, RAM-bound," "keys are integers with bounded range," "must be stable."

Looking ahead. Chapters 4-6 spend their entire budget building two or three of the structures from the list here (linked lists, hash tables, BSTs). Every performance claim in those chapters is phrased in Big-O, and most of the sort algorithms in 3.15-3.21 reappear as components (e.g., merge sort is the

model for merging sorted linked lists in ch. 4).

1 3.1 Representing Information as Bits

Chapter 2 covered algorithms – the *instructions* that process data. Chapter 3 shifts the view to how data is *represented* in the first place. At the bottom of every stack, "data" is ultimately a sequence of tiny binary switches. This section is the ground-floor view: what a bit is, what a byte is, and how characters like `'A'` or `'Z'` get mapped onto patterns of bits.

1.1 Bits and bytes

Definition — Bit

A **bit** ("binary digit") is the smallest unit of information: a single `0` or `1`. Physically, it corresponds to a switch, a charged capacitor, a magnetic polarity, or a voltage level – anything with two stable states.

Definition — Byte

A **byte** is a group of **8 bits**, giving $2^8 = 256$ distinct values (bit patterns `00000000` through `11111111`). Memory, file sizes, and network transfers are almost always measured in bytes or larger units (KB, MB, GB ...).

Aside Nibbles, halfwords, and other groupings

A **nibble** is 4 bits (half a byte). A **word** is typically what the CPU processes in one step – 4 bytes (32-bit) or 8 bytes (64-bit) on modern machines. C++ types you've already used map onto these sizes: `char` is 1 byte, `int` is usually 4, a pointer or `size_t` on a 64-bit machine is 8. `sizeof(T)` tells you the number of bytes.

1.2 Why bits

Every piece of data – an integer, a character, a pixel, a video frame, a neural network's weights – gets ultimately stored as a pattern of bits. The question isn't *whether* data is bits but *which encoding* the program uses to interpret those bits. Two different encodings can give the same pattern two different meanings:

Example `01000001` – what does it mean?

- As an unsigned 8-bit integer: 65.
- As a signed 8-bit integer (two's complement): 65.
- As an ASCII character: `'A'`.
- As a pair of nibbles: 4 and 1.
- As a bitfield: bit 6 on, bits 0 and 5 on, rest off.

Same eight bits, five different interpretations. The program decides which one applies based on the type declaration.

1.3 ASCII: bits that mean characters

Definition — Character encoding

A **character encoding** is a table mapping numeric codes to characters (letters, digits, punctuation, symbols). To "store a string," the computer stores the sequence of code values; when it "prints a string," it looks up the characters those codes represent.

Definition — ASCII

ASCII (*American Standard Code for Information Interchange*) is the oldest widely-used character encoding. It assigns a unique 7-bit code (values `0`–`127`) to each printable English character plus a set of control codes. Because it fits in 7 bits, every ASCII character fits comfortably in a single byte with one bit to spare.

1.4 A subset of the ASCII table

The full ASCII table is 128 entries. A few useful patterns to commit to memory:

Range (dec)	Range (hex)	Meaning
0–31	`0x00`–`0x1F`	Control characters (non-printing)
32	`0x20`	Space
33–47	`0x21`–`0x2F`	Punctuation: `! " # $ % & ' ( ) * + , - . /`
48–57	`0x30`–`0x39`	Digits `'0'`–`'9'`
58–64	`0x3A`–`0x40`	More punctuation: `: ; < = > ? @`
65–90	`0x41`–`0x5A`	Uppercase `'A'`–`'Z'`
91–96	`0x5B`–`0x60`	`[ \ ] ^ _ ‘`
97–122	`0x61`–`0x7A`	Lowercase `'a'`–`'z'`
123–126	`0x7B`–`0x7E`	`{ | } ~`
127	`0x7F`	DEL (control)

Key idea The three patterns worth remembering

1. `'0'` is 48, `'1'` is 49, …, `'9'` is 57. **`c - '0'`** converts a digit character to its integer value.
2. `'A'` is 65, `'Z'` is 90; `'a'` is 97, `'z'` is 122. Uppercase and lowercase differ by exactly **32** (2^5), so `c + 32` lowercases an uppercase letter and `c - 32` uppercases a lowercase one. `c ^ 32` does both (toggle case).
3. Space is 32 = $0x20$. Every printable character is $\geq$ 32. Anything below is a control character (newline is 10, carriage return is 13, tab is 9, null is 0).

These three facts alone cover half of the character-arithmetic bugs you'll hit.

Example Digit-char to int, by hand

```
char c = '7';
int  d = c - '0';     // 55 - 48 == 7

// Accumulate digits from a string into a number:
int n = 0;
for (char ch : s) {
    if (std::isdigit(ch)) n = n * 10 + (ch - '0');
}
```

Example Case conversion without cctype

```
char upper = 'g' - 32;     // 'G'
char lower = 'G' + 32;     // 'g'
char flip  = 'G' ^ 32;     // 'g'
char flip2 = 'g' ^ 32;     // 'G'
```

Using `std::toupper`/`std::tolower` is still the right production choice – they handle the non-letter case and locale concerns correctly – but knowing the underlying bit trick helps when you read code that uses it.

1.5 What ASCII can't do: everything else

ASCII encodes English. It has no code points for accented letters (é, ñ), no space for non-Latin scripts (Arabic, Chinese, Cyrillic, Tamil), and no emoji. The modern universal encoding is **Unicode**, which assigns code points up to $10FFFF_{16}$ (~ 1.1 million characters).

Definition — Unicode and UTF-8

Unicode is the catalog of characters and their numeric code points. **UTF-8** is the most common byte-level encoding: ASCII characters still fit in one byte (and use the same values as ASCII!), while higher code points use 2–4 bytes. UTF-8 is backward compatible with ASCII, which is why almost every file, web page, and API defaults to it.

Gotcha `char` vs. Unicode in C++

A `char` in C++ is one byte. A UTF-8 character (like é) can occupy multiple bytes. Code that walks a UTF-8 string one `char` at a time is counting bytes, not characters, and treating each byte as an independent character will produce garbage output and wrong indices for any non-ASCII input. C++ offers `char16_t`, `char32_t`, and `std::u8string` for wider character work, plus libraries like ICU for correct text handling.

For CS 300 projects with English-only input you rarely need to worry – but the assumption "1 char = 1 character" is only true for ASCII, and "is only true for ASCII" means it's only true for one small slice of the world's text.

1.6 Connecting bits back to data structures

Every data structure you're about to meet (linked list, stack, queue, tree, hash table) is ultimately a layout of bits in memory, organized so that the operations the structure promises are fast. A linked list stores a value plus a pointer-to-next-node; a vector stores an array of values plus a size and capacity; a hash table stores an array of buckets plus a hash function. The abstractions hide the bits, but understanding that the bits are still there – and that performance ultimately comes from how they're arranged – is what separates "using" a data structure from *choosing* one.

Key idea Big picture

All data is bits; all bit patterns need an encoding to become data. ASCII is the oldest and simplest encoding, covering exactly the characters on a 1960s American typewriter, and is still the foundation of modern text encodings that extend it to the rest of human writing. The arithmetic shortcuts – `c - '0'` for digits, `^ 32` for case, `c >= '0' && c <= '9'` for "is digit" – come from ASCII's tidy layout and show up constantly in real parsing code. Build intuition for these now; every chapter from here on assumes it.

2 3.2 Data Structures

A **data structure** is a way of organizing data in memory so that a particular set of operations (access, search, insert, delete) can be done efficiently. §3.1 looked at how individual values are stored as bits; this section starts naming the ways those values get grouped into useful collections. The rest of the course is essentially a detailed tour of the seven structures listed here.

Definition — Data structure

A **data structure** is a combination of a *layout in memory* and a *set of operations* that together give certain common tasks good performance characteristics. The same data can be organized many ways – the choice depends on which operations matter most for your workload.

2.1 The seven structures you'll meet this semester

Definition — Record (struct, class)

A group of named **fields** bundled into a single value. Fields can be of different types. In C++, a `struct` or `class` is a record.

```
struct Bid {
    std::string title;
    std::string fund;
    double amount;
};
```

Definition — Array / vector

An **ordered** list of items stored in *contiguous memory*, directly indexable by position. Access is $O(1)$; insert/delete in the middle is $O(n)$ because later items have to shift.

Definition — Linked list

An ordered list stored as a chain of **nodes**, each holding a value and a pointer to the next node. No contiguous memory, no indexing. Insert/delete given a pointer to the node is $O(1)$; accessing the k-th item is $O(k)$ because you have to walk the chain.

Definition — Binary tree

A hierarchical structure where each node has at most two children (left and right). The foundation for binary search trees, heaps, expression trees, and more. Good implementations give $O(\log n)$ search, insert, and delete on balanced variants.

Definition — Hash table

An array plus a **hash function** that maps keys to array positions. Average-case $O(1)$ for insert, lookup, and delete; no ordering. The standard choice when you need fast keyed access and don't need sorted iteration.

Definition — Heap

A tree maintaining the **heap property**: in a *max-heap*, each node's key is $\geq$ its children's keys; in a *min-heap*, $\leq$. Gives $O(\log n)$ insert and remove-extreme. The underlying structure behind priority queues and heapsort.

Definition — Graph

A collection of **vertices** connected by **edges**, used to model networks (roads, social graphs, dependencies, web links). Can be directed or undirected, weighted or unweighted. Operations include traversal, shortest path, and connectivity.

2.2 Why there are so many

Every structure makes different operations fast at the cost of others. There is no "best" data structure; the right one depends on the operation mix you actually perform.

Structure	Access	Insert mid	Good for
Array / vector	$O(1)$	$O(n)$	Random access, cache-friendly
Linked list	$O(n)$	$O(1)^*$	Frequent insert/delete
Hash table	$O(1)$ avg	$O(1)$ avg	Fast keyed lookup
Binary search tree	$O(\log n)^\dagger$	$O(\log n)^\dagger$	Sorted iteration
Heap	—	$O(\log n)$	Always know the max/min
Graph	—	varies	Relationships, networks

*Given a pointer to the insertion point. Finding the point is still $O(n)$.

†Balanced only. A degenerate BST is $O(n)$.

2.3 A concrete example: the shift problem

The classic case where structure choice matters is inserting in the middle of a collection.

Example Array insertion is $O(n)$

Start: `[A, C, D]`. Insert `B` at position 1.
To make room, `C` and `D` have to shift right: `[A, _, C, D]` $\rightarrow$ `[A, _, _, C, D]` $\rightarrow$ `[A, B, C, D]`.
Every item after the insertion point moves. For an array of size n, worst-case insertion moves n items.

Example Linked list insertion is $O(1)$ (given the node)

Start: `A` $\rightarrow$ `C` $\rightarrow$ `D`. To insert `B` between `A` and `C`:

1. Allocate a new node `B`.
2. Point `B.next` at the node `C`.
3. Point `A.next` at `B`.

Two pointer writes. No shifting, no matter how long the list is.

Gotcha "$O(1)$ linked list insert" has a hidden assumption

The $O(1)$ cost assumes you already have a pointer to the node where you want to insert. If you have to *find* that node first – "insert B after the node containing A" – you pay $O(n)$ to find it, then $O(1)$ to do the insert. For insertion at an arbitrary position by value, linked lists usually don't beat vectors in practice.

2.4 How to pick a data structure

Key idea A practical decision procedure

1. **What operations dominate?** Access by index, by key, by position? Insert at the ends, or in the middle? Iterate in sorted order? Always need the min/max?
2. **What's the expected size?** Small (≤ 100) almost anything is fine and a vector usually wins by cache alone. Large means the asymptotic cost matters.
3. **What invariant do you need?** Sorted? Unique? Order of insertion preserved? That usually rules out several structures.
4. **Is the standard-library version good enough?** `std::vector`, `std::unordered_map`, `std::set`, and `std::priority_queue` cover most needs – reach for a custom structure only when measurement shows the defaults are the bottleneck.

Example Mapping three typical CS 300 tasks to structures

- "Look up a course by its ID." → hash table (`std::unordered_map<std::string, Course>`).
- "Print courses in alphabetical order." → balanced BST (`std::map`) or a vector + sort once at the end.
- "Process bids in arrival order." → queue (backed by a linked list or a ring buffer).

Same data, different access pattern, different structure.

2.5 The C++ standard library's version of this list

Almost every structure named above already ships with the language. Know the mapping so you don't reinvent:

Concept	C++ standard type
Array / vector	`std::vector<T>`, `std::array<T, N>`
Linked list	`std::list<T>` (doubly) or `std::forward_list<T>`
Hash table	`std::unordered_map<K, V>`, `std::unordered_set<T>`
Balanced BST	`std::map<K, V>`, `std::set<T>`
Heap / priority queue	`std::priority_queue<T>`
Stack / queue (adapters)	`std::stack<T>`, `std::queue<T>`
Graph	no built-in – build from `vector<vector<int»` or similar

Aside Build your own for understanding, use the STL in practice

CS 300 will have you implement versions of several of these from scratch. That's for learning. In real code, reach for the library first; it's tuned, tested, and portable. Understanding what's happening inside `std::unordered_map` makes you better at using it, not a replacement for using it.

Key idea Big picture

The rest of the course is a depth-first walk through these seven structures – their implementations, their asymptotic costs, and the patterns of problem each one fits. Think of this section as the map: every later chapter picks one structure, builds it in detail, analyzes its operations, and shows the problems it solves. When you finish, the skill is not "I've memorized all seven" but "I can look at a problem and say, that's a bag-of-key-value pairs with fast lookup – hash table."

3 3.3 Relation Between Data Structures and Algorithms

Chapter 2 treated algorithms in the abstract; section 3.2 laid out the main data structures. This section is the hinge between them. The relationship runs in *two directions*, and keeping both in mind is what turns "I know some algorithms" and "I know some structures" into "I can design software."

Key idea Two directions of the same relationship

1. **Algorithms *for* a data structure.** Each structure needs its own implementation of common operations (insert, remove, search). The *same operation name* hides very different algorithms depending on the underlying structure.
2. **Algorithms that *use* data structures.** A larger algorithm typically leans on one or more structures as scratch space – somewhere to accumulate, sort, or look up data while the algorithm runs.

3.1 Direction 1: Different structure, different algorithm

"Append an item" sounds like one operation, but the code behind it depends entirely on what you're appending to. Compare appending to a linked list against appending to an array.

Example Append to a singly linked list – `O(1)`

```
void append(Node*& head, Node*& tail, int value) {
    Node* n = new Node{value, nullptr};
    if (!head) {            // empty list
        head = tail = n;
    } else {
        tail->next = n;     // stitch onto the end
        tail = n;
    }
}
```

With a tail pointer cached, linking on a new node is a couple of pointer writes – no elements moved, no memory copied.

Example Append to a fixed-capacity array – `O(1)` amortized, `O(n)` worst case

```
void append(int*& data, int& size, int& cap, int value) {
    if (size == cap) {
        int newCap = cap == 0 ? 1 : cap * 2;
        int* grown = new int[newCap];
        for (int i = 0; i < size; ++i) grown[i] = data[i];  // copy all
```

```
        delete[] data;
        data = grown;
        cap = newCap;
    }
    data[size++] = value;
}
```

Most appends just drop the value into the next slot. But whenever the array fills up, every existing element has to be copied into a larger block. `std::vector` hides this dance behind `push_back`, but the copy still happens.

Aside "Amortized" in one sentence

If doubling on overflow makes some individual pushes expensive but *averages out* to constant time across many pushes, we say `push_back` is **amortized** $O(1)$. You pay a big cost occasionally, but the cost per call, averaged over a long run, stays constant. We'll make this precise when we cover analysis.

Insert-at-front is the sharper example

Append hides a lot because both versions *usually* finish fast. Prepend makes the structure choice visible immediately.

Operation	Array / `vector`	Singly linked list
Append at end	$O(1)$ amortized	$O(1)$ with tail pointer
Insert at front	$O(n)$ (shift all)	$O(1)$ (new head)
Random access `a[i]`	$O(1)$	$O(i)$ (walk from head)
Insert in middle (given iterator)	$O(n)$ shift	$O(1)$ relink

Same *operation names*, completely different runtimes. The structure you pick *is* the set of costs you accept.

Gotcha "Same name" is misleading

Don't trust operation names across containers. `push_back` on a `std::vector` is fast; `push_front` doesn't even exist (use `std::deque` or `std::list`). `find` on `std::vector` is $O(n)$; on `std::unordered_map` it's expected $O(1)$. Always ask: what is the cost of this operation *on this structure*?

3.2 Direction 2: Algorithms use structures as scratch space

The other direction: a larger algorithm almost always relies on a data structure to hold intermediate results. The structure you pick shapes the algorithm you write.

Example: top-k salespersons. Given a list of salespeople, print the top 5 by sales total.

Example A small fixed-size array as running top-5

```
struct Rep { std::string name; double sales; };

void topFive(const std::vector<Rep>& all) {
    std::array<Rep, 5> top{};
    for (auto& r : top) r.sales = -1;        // sentinel
```

```
    for (const Rep& r : all) {
        if (r.sales > top.back().sales) {    // beats last in top-5
            top.back() = r;
            std::sort(top.begin(), top.end(),
                      [](const Rep& a, const Rep& b){
                          return a.sales > b.sales;
                      });
        }
    }
    for (const Rep& r : top) std::cout << r.name << " " << r.sales << "\n"
        ;
}
```

The fixed-size array is the data structure; the scan-and-swap pattern is the algorithm. The array's job is to remember the current champions as we sweep through all candidates.

Key idea Why fixed-size-5 instead of "sort them all"?

Sorting the entire list is $O(n \log n)$. The running-top-5 sweep is $O(n \log k)$ where $k = 5$ – effectively $O(n)$ because the inner sort is on a constant-size array. For top-k problems where $k \ll n$, this is the standard trick. (A heap of size k is even better; we'll see that in the priority-queue chapter.)

Different structure, different algorithm

If we used a different structure to hold the running top-5, the algorithm changes shape:

- **Min-heap of size 5.** Peek at the minimum in $O(1)$; if the new candidate beats it, pop and push. Total $O(n \log k)$ but cleaner and faster than re-sorting an array.
- **Sorted linked list of size 5.** Insert-in-order means walking the list to find the right slot – $O(k)$ per candidate that beats the minimum.
- **BST / `std::set`.** Overkill here, but useful if k were large or you needed dynamic queries.

Same problem, four different implementations, each dictated by the structure you choose.

Gotcha Structure choice $\neq$ premature optimization

"Pick the simplest thing that works" is good advice. But the structure is often what determines whether "works" is even feasible at your data size. A naive $O(n^2)$ search that's *correct* is fine for 100 items and ruinous for 100 000 000. The choice is part of the design, not a polish step at the end.

3.3 Putting the two directions together

Key idea What it means to "design" something

When you sit down with a new problem, the workflow is:

1. **What data does the algorithm need access to, and how?** By index? By key? In sorted order? First-in-first-out?
2. **Which structure answers those access patterns cheaply?** Consult the table from 3.2.

3. **Now: what does the algorithm look like given that structure's operations?** Its cost is the sum of the per-step costs in the structure you picked.

Step 2 is the choice *of* structure; step 3 is the algorithm *using* it. Those are the two directions, in order, every time.

Aside Where this is going

The rest of the course iterates this loop structure-by-structure. For each of the seven structures from 3.2, we'll define it precisely, implement it in C++, analyze the cost of its core operations, and study the algorithms that either implement it or use it. By the end, the "design" step above will be muscle memory.

4 3.4 Abstract Data Types

The previous section showed that the *same operation name* on two different data structures hides two different algorithms. That's a feature, not a bug. The whole point of this section's central idea – the **abstract data type** – is to let us *name an operation without naming an implementation*, so code that uses the operation keeps working no matter which structure sits underneath.

Definition — Abstract data type (ADT)

An **abstract data type** is a data type defined by the *operations* it supports and the *behavior* of those operations, **without** specifying how they are implemented. An ADT is a contract: "you can do these things, and they mean this." A **data structure** is one concrete way to honor that contract.

Key idea ADT vs. data structure, in one line

An **ADT** is *what*. A **data structure** is *how*. A single ADT can have many structures behind it; a single structure can be used to implement several ADTs.

4.1 Why the separation matters

Everyday analogy: "printer" is an ADT (print this page, cancel this job). Laser printer, inkjet, thermal, network printer – all different implementations. The program calling `print()` does not and should not care. Swap the printer, the program still works.

In code, the benefits land as:

- **Replaceability.** Start with an array-backed list; switch to a linked-list-backed list when profiling tells you to. No caller code changes.
- **Reasoning.** The correctness of caller code depends on the ADT's *contract*, not on representation details.
- **Teamwork.** One person builds the ADT, another builds against its interface. They negotiate a contract, not a memory layout.

Gotcha Contract includes *cost*

A pure contract-of-behavior is not quite enough in this course. When we promise "$O(1)$ lookup," the constant-time guarantee is part of the ADT's contract. Two implementations that both return the right answer can still be usefully different ADTs if their cost profiles differ – which is why `std::map` (balanced BST, $O(\log n)$) and `std::unordered_map` (hash table, expected $O(1)$) are separate types rather than one "dictionary" type with a hidden switch.

4.2 The standard ADTs you should recognize

The textbook lists nine common ADTs. Treat this table as a vocabulary list – each one gets its own chapter later.

ADT	What the contract says	Usual implementations
List	Ordered sequence; append, remove, search, traverse.	Array, linked list
Dynamic array	Ordered with $O(1)$ indexed access; grows on demand.	Array (with resize)
Stack	LIFO; push / pop / peek at the top only.	Array, linked list
Queue	FIFO; enqueue at back, dequeue from front.	Linked list, circular array
Deque	Insert/remove at *both* ends.	Doubly linked list, circular array
Bag	Unordered collection; duplicates allowed.	Array, linked list
Set	Unordered; *no* duplicates.	BST, hash table
Priority queue	Queue where highest-priority item comes out first.	Heap
Dictionary / Map	Key $\rightarrow$ value lookup.	Hash table, BST

Aside How to read the rows

Two axes usually disambiguate them: (1) is order required, and if so what kind (insertion order, priority order, sorted by key)? (2) are duplicates allowed? A *bag* has duplicates and no order; a *set* has no duplicates and no order; a *list* has duplicates and insertion order; a *priority queue* has duplicates and priority order.

4.3 C++ makes the ADT/structure distinction concrete

The C++ standard library names ADTs via *container adaptors* that sit on top of underlying containers. This is worth seeing explicitly because it's the same idea the textbook is describing, except made literal in the type system.

Example `std::stack` is an ADT built on another container

```
#include <stack>
#include <vector>
#include <list>

std::stack<int>                      s1;  // default: uses std::deque
std::stack<int, std::vector<int>>    s2;  // uses a vector
std::stack<int, std::list<int>>      s3;  // uses a linked list
```

```
s1.push(3); s1.push(4);
int top = s1.top();  // 4
s1.pop();
```

All three behave identically from the caller's point of view: a stack. What differs is the underlying storage. The ADT (`stack`) is the interface; the structure is a template parameter.

The library's ADT / structure mapping

ADT (from textbook)	C++ name	Default structure
List	`std::list`, `std::forward_list`	Doubly / singly linked list
Dynamic array	`std::vector`	Contiguous array
Stack	`std::stack`	`std::deque`
Queue	`std::queue`	`std::deque`
Deque	`std::deque`	Chunked array
Set	`std::set`, `std::unordered_set`	BST / hash table
Priority queue	`std::priority_queue`	Binary heap over `std::vector`
Map	`std::map`, `std::unordered_map`	BST / hash table

(C++ doesn't ship a "bag" type; a `std::vector` or `std::multiset` fills that role.)

Gotcha Don't confuse `std::vector` and "array"

C++'s built-in array (`int a[10]`) is a fixed-size data structure with no resize. `std::vector` is the ADT "dynamic array," implemented *using* a C array plus a size and capacity counter. When someone says "array" in conversation about C++, they usually mean `std::vector`. Read carefully.

4.4 Using ADTs to keep caller code honest

Example Algorithm expressed against an ADT

```
// Reverse the order of items in any container via a stack.
template <typename Container>
void reverseInPlace(Container& c) {
    std::stack<typename Container::value_type> buf;
    for (auto& x : c) buf.push(x);   // push all
    for (auto& x : c) {              // pop back into place
        x = buf.top();
        buf.pop();
    }
}
```

This works for any container. The algorithm depends only on the *stack* ADT – push, top, pop. Whatever storage `std::stack` uses, the reverse works.

Key idea The ADT-first habit

When you're sketching a solution, name the ADT you need ("I need a priority queue") *before* you pick a structure ("I'll back it with a binary heap"). Most design mistakes at this level come from picking a

structure first, then contorting the algorithm around it. The fix: force yourself to say the ADT out loud first.

Aside Where this is going

Starting in Chapter 4, every chapter for the rest of the course follows the same pattern: pick an ADT, pick a structure (or two) that implement it, build those implementations in C++, and analyze each operation's cost. Sections 3.2 – 3.4 gave you the vocabulary to recognize which ADT/structure pairing is in front of you at any given moment.

5 3.5 Applications of ADTs

Section 3.4 made the ADT/structure distinction; this section asks what you *get* from keeping them separate. Two payoffs: **abstraction** simplifies programming, and **standard libraries** mean you rarely implement the common ADTs by hand. The catch – which this section also makes explicit – is that abstraction does not free you from thinking about efficiency.

5.1 Abstraction as a productivity lever

Definition — Abstraction

Abstraction is the practice of interacting with something through a *high-level interface*, with lower-level details hidden behind that interface. ADTs are one concrete application of the idea: the operations are the interface, the structure is hidden.

Abstraction buys three compounding benefits once you're comfortable with it:

- **Less code to write.** You describe *what* you want, not *how* to do it. `map[key] = value` hides a hash computation, bucket lookup, collision handling, and possible rehash.
- **Less code to read.** A colleague scanning your code sees the ADT's operations and immediately knows the shape of the data flow – "oh, it's a queue, so FIFO" – without studying a custom linked-list implementation.
- **Less code to debug.** Bugs in the ADT's implementation live in one place; bugs in your algorithm live in yours. The two don't mix.

Example Same job, two abstraction levels

```
// Low-level: roll our own LIFO using a raw array + top index.
int stack[100];
int top = -1;
stack[++top] = 3;
stack[++top] = 4;
int x = stack[top--];          // 4
int y = stack[top--];          // 3

// High-level: lean on the stack ADT from the STL.
#include <stack>
std::stack<int> s;
```

```
s.push(3);
s.push(4);
int x = s.top(); s.pop();        // 4
int y = s.top(); s.pop();        // 3
```

Both snippets do the same thing. The first mixes the algorithm with the storage. The second talks only in stack operations – push, top, pop – so a reader grasps the intent in one pass.

Gotcha Abstraction doesn't erase cost

The textbook is explicit about this: *"knowledge of the underlying implementation is needed to analyze or improve the runtime efficiency."* A `std::map` and a `std::unordered_map` expose the same dictionary ADT, but one is $O(\log n)$ and the other is expected $O(1)$ per operation – with different worst-case behavior, cache profiles, and memory overhead. When performance matters, the structure matters. Pretending otherwise is how teams end up with "mysteriously slow" code.

5.2 ADTs in standard libraries

Almost every modern language ships an *implementation* of the common ADTs as part of its standard library. You rarely implement a queue or a hash map from scratch in production; you use the language's provided one.

Language	Library	ADTs you get
C++	Standard Template Library (STL)	`vector`, `list`, `deque`, `stack`, `queue`, `priority_queue`, `set`, `map`, `unordered_set`, `unordered_map`
Python	Built-ins + `collections`	`list`, `tuple`, `set`, `dict`, `deque`, `defaultdict`, `Counter`, `heapq`
Java	Java Collections Framework (JCF)	`ArrayList`, `LinkedList`, `HashSet`, `TreeSet`, `HashMap`, `TreeMap`, `ArrayDeque`, `PriorityQueue`

Aside How much do languages let you choose?

Languages differ in how much control they give you over the underlying structure:

- **C++** is the most explicit. Adaptors like `std::stack` and `std::priority_queue` accept the underlying container as a template parameter; separate types (`map` vs `unordered_map`, `list` vs `forward_list`) make the choice a deliberate decision.
- **Python**'s `list` is always a dynamic array; `dict` is always a hash table. The runtime picks; you don't.
- **Java** exposes interface-vs-implementation explicitly: you declare a variable as `List<String>` and assign it either an `ArrayList` or a `LinkedList`.

5.3 Why CS 300 still has you build them by hand

Production code uses the standard library. This course has you implement linked lists, hash tables, heaps, and trees from scratch. That's deliberate, and for two reasons.

Key idea Building vs. using

1. **Understanding.** To reason about performance you have to know what is happening under the interface. Implementing a hash table once makes expressions like "good hash function," "load factor,"

and "worst-case $O(n)$ on degenerate input" stop being vocabulary and start being intuition.

2. **Design judgment.** Choosing between `std::map` and `std::unordered_map` in a real project requires knowing what each has to do internally. Library knowledge without structural knowledge leads to mysterious performance cliffs.

Gotcha In real projects, reach for the library first

Outside this course, re-implementing a hash map is a bright red flag in a code review. The standard library's versions are tuned over decades, heavily tested, and portable. Build-your-own is for *learning the idea*, not *shipping the product*.

5.4 Choosing the right ADT is the actual skill

Because the common ADTs are available off the shelf, the hard part isn't implementing them – it's recognizing which one the problem in front of you is asking for.

Problem shape	ADT
"I need the most recent item first."	Stack
"I need the oldest waiting item first."	Queue
"I need the *most important* item first."	Priority queue
"I need fast lookup by a key."	Map / dictionary
"I need a collection of distinct items, order doesn't matter."	Set
"I need ordered access by index."	Dynamic array / list

Aside Where this is going

This closes the "setup" arc of the course. Chapter 4 starts the walk: pick an ADT or structure, build it, analyze it, use it. The background from 3.1–3.5 – bits, the seven structures, the algorithm / structure relationship, ADTs, and standard libraries – is the common language every later chapter speaks.

6 3.6 Searching and Algorithm Runtime

This section restarts the algorithms thread now that we have a vocabulary of data structures to search through. The worked example is *linear search*, the simplest search algorithm there is. The real payoff of the section, though, is the first careful look at **runtime as a function of input size** – the idea that turns into Big-O in §3.10.

6.1 Linear search

Definition — Linear search

Linear search (also called sequential search) walks a collection from the first element to the last, comparing each element to a search key, returning the index where it matches or a sentinel value (often `-1`) if it never does.

Example Linear search in C++

```
int linearSearch(const std::vector<int>& nums, int key) {
    for (std::size_t i = 0; i < nums.size(); ++i) {
        if (nums[i] == key) {
            return static_cast<int>(i);
        }
    }
    return -1;  // not found
}

int main() {
    std::vector<int> nums = {2, 4, 7, 10, 11, 32, 45, 87};
    int idx = linearSearch(nums, 10);
    if (idx == -1) std::cout << "not found\n";
    else           std::cout << "found at index " << idx << '\n';
}
```

Aside STL already has it

`std::find(v.begin(), v.end(), key)` returns an iterator to the first matching element, or `v.end()` if none. Same algorithm, idiomatic C++. Hand-rolling the loop is a learning exercise; in production, use the library.

6.2 Why linear – and when it's fine

Linear search makes *no* assumptions about the data. It works on an unsorted array, a linked list, or any sequence you can iterate once. That's the whole point: it's the baseline.

- **Best case.** Key is at index 0 – **1 comparison**.
- **Worst case.** Key is last, or absent – n **comparisons**.
- **Average case.** On uniformly random hit positions, roughly $n/2$ comparisons. When the key is absent, always n.

Gotcha Linear search on a linked list is still $O(n)$

Don't confuse *structure* with *algorithm*. Linear search is $O(n)$ whether the container is a `std::vector` or a `std::list`. The linked list's poor random access doesn't make linear search any worse – linear search only ever moves to the *next* element, which is cheap for both structures.

6.3 Runtime as a function of input size

The key idea the textbook introduces here: runtime grows with the size of the input. Counting *actual seconds* depends on the machine. Counting *the number of operations the algorithm does* is a machine-independent proxy that predicts the seconds.

Key idea Order of growth, informally

If an algorithm does "about N" operations on an input of size N, we say its runtime is **on the order of** N. Doubling the input doubles the work. Linear search is on the order of N; we'll soon call this $O(N)$.

Concrete numbers

The scale at which "$O(N)$" starts to feel slow surprises people the first time they see it. Suppose one comparison takes 1 microsecond (10^{-6} s).

N	Worst-case comparisons	Wall time @ 1 µs each
10^3	1,000	0.001 s
10^6	1,000,000	1 s
10^7	10^7	10 s
2×10^8 (Amazon's catalog, ish)	2×10^8	200 s $\approx$ 3 min 20 s
10^9	10^9	about 17 min

Gotcha "Fast enough" is always "fast enough *for what?*"

1 second is fine if you're searching once. For an interactive search box that fires on every keystroke, it's catastrophic. For a background indexer, it may be fine. *Runtime is a constraint, not a verdict – what matters is whether it fits the use case.*

6.4 Ignoring constants: why we say "on the order of N"

If your linear search does 3 operations per element (bounds check, array access, comparison) instead of 1, the constant changes but the *shape* of the growth does not: doubling N still doubles the work.

Key idea The shape is what matters

For large N, the *shape* of the growth (constant, linear, quadratic, logarithmic …) dominates any constant-factor difference. An $O(N)$ algorithm with a big constant will eventually beat an $O(N^2)$ algorithm with a small one. "Eventually" may be sooner than you think: at $N = 10{,}000$, a $100N$ algorithm does 10^6 ops; a $0.1N^2$ algorithm does 10^7. The shape has already won.

6.5 Searching, structures, and algorithms together

The textbook frames this section as "algorithms," but notice how much depends on the structure underneath. Linear search is the universal baseline because it makes no assumptions. If you know more about the structure, you can do much better:

What you know about the data	Algorithm	Worst case
Nothing; unsorted sequence	Linear search	$O(N)$
Array is sorted	Binary search	$O(\log N)$
Data lives in a hash table keyed by what you're searching for	Hash lookup	$O(1)$ expected
Data lives in a balanced BST keyed by the search value	BST lookup	$O(\log N)$

Aside Where this is going

§3.10 formalizes the "on the order of" idea into Big-O notation, and §3.8 introduces binary search – the first algorithm that does substantially better than $O(N)$, but only on a *sorted* structure. That's the cycle you'll see again and again: pick an algorithm, pay the structure's cost; pick a structure, unlock a better algorithm.

7 3.8 Binary Search

Section 3.6 covered linear search – the universal baseline. This section introduces the first algorithm that does substantially better: **binary search**. The catch, and the whole point, is that it only works if the data is *sorted and randomly accessible*. That trade-off – accept a precondition on the structure, unlock a much faster algorithm – is a pattern you will see repeatedly for the rest of the course.

7.1 The idea

Think about how you find a name in a physical phone book. You don't read from the beginning. You open near the middle, see whether your target comes before or after, and immediately ignore the half that doesn't contain it. Repeat.

Definition — Binary search

Binary search finds an item in a *sorted, randomly accessible* sequence by repeatedly looking at the middle element and discarding the half that cannot contain the key. Each step halves the search space; the algorithm terminates when the key is found or the space is empty.

Gotcha Two preconditions – both matter

Binary search requires both:

- **Sorted.** Without order, "look at the middle" tells you nothing about which half the key is in.
- **Random access in $O(1)$.** You have to jump straight to the middle. On an array, that's `a[mid]`. On a *linked list*, reaching the middle takes $O(n)$ – and that extra cost destroys the $O(\log n)$ advantage.

So binary search on a `std::list` is *not* a thing; on `std::vector` (or a plain array), it is.

7.2 Binary search in C++

Example Iterative binary search

```
int binarySearch(const std::vector<int>& nums, int key) {
    int low = 0;
    int high = static_cast<int>(nums.size()) - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;   // overflow-safe midpoint
        if      (nums[mid] == key) return mid;
        else if (nums[mid]  < key) low  = mid + 1;
        else                       high = mid - 1;
    }
    return -1;  // not found
```

```
}
```

The textbook writes `(low + high) / 2`. That formula can overflow when `low + high` exceeds `INT_MAX`. The idiomatic fix is `low + (high - low) / 2`, which computes the same midpoint without the intermediate sum blowing up.

Aside Real-world gotcha, not a hypothetical

This is a famous bug. JDK's `Arrays.binarySearch` had the naïve formula for nearly a decade before Josh Bloch documented it ("Extra, Extra – Read All About It: Nearly All Binary Searches and Mergesorts are Broken," 2006). Use the overflow-safe form.

Recursive binary search

Binary search is also the textbook example of a recursive algorithm that calls itself on a *shrinking* problem.

Example Recursive binary search

```
int binarySearch(const std::vector<int>& nums,
                 int key, int low, int high) {
    if (low > high) return -1;                  // base case
    int mid = low + (high - low) / 2;
    if (nums[mid] == key) return mid;
    if (nums[mid]  < key) return binarySearch(nums, key, mid + 1, high);
    return                    binarySearch(nums, key, low, mid - 1);
}
```

Same algorithm; two views. The iterative version updates a window; the recursive version calls itself on a smaller window. The iterative version is usually preferred in C++ because it avoids stack frames.

7.3 Why $O(\log n)$

Key idea The halving argument

Start with n candidates. After one step, at most $n/2$. After two steps, $n/4$. After k steps, $n/2^k$. The loop ends when the window is empty; that happens when $n/2^k \leq 1$, i.e. when $k \geq \log_2 n$.
Binary search does at most

$$\lfloor \log_2 n \rfloor + 1$$

comparisons in the worst case. The runtime is $\boldsymbol{O(\log n)}$.

Why $\log n$ is so dramatic

The concrete comparison with linear search is what sells binary search. One comparison at 1 µs:

N	Linear worst	Binary worst	Ratio
10^3	1,000 ops (1 ms)	10 ops (10 µs)	100×
10^6	10^6 ops (1 s)	20 ops (20 µs)	50,000×
2×10^8	2×10^8 (3+ min)	28 ops (28 µs)	$\approx 7 \times 10^6\times$
10^9	10^9 ops (≈17 min)	30 ops (30 µs)	$\approx 3 \times 10^7\times$

Key idea The "log bar" for intuition

$\log_2 n$ is the number of times you can halve n before reaching 1. For any n up to a billion, that's at most 30. Double the input, you add *one* operation. This is the first example in the course of truly different growth behavior – the gap between linear and logarithmic is not "a bit faster," it's categorical.

7.4 The bigger lesson

Gotcha Sorting isn't free

If your data isn't already sorted, you have to sort it first – which is $O(n \log n)$ for the first sort. One binary search on a freshly sorted array does not beat one linear search. Binary search pays off when you will do *many* searches against the same sorted data, amortizing the sort cost across the searches.

Key idea Structure + precondition = algorithm

This is the first of many "precondition unlocks algorithm" patterns you will meet. The full arc:

- *Unsorted sequence.* Linear search, $O(n)$.
- *Sorted sequence, random access.* Binary search, $O(\log n)$.
- *Binary search tree.* Tree descent, $O(\log n)$ expected.
- *Hash table keyed by the search value.* Direct lookup, $O(1)$ expected.

Each improvement comes from structuring the data to answer the question you'll ask. Algorithm design is in large part the art of preparing the data so that lookup is cheap.

Aside STL ships binary search

`std::binary_search(v.begin(), v.end(), key)` returns a `bool`. `std::lower_bound` returns the iterator to the first element $\geq$ key (where you'd insert). You'll use these a lot when a sorted structure is the right choice.

8 3.9 Constant-Time Operations

Binary search was the first algorithm where the *number of operations* mattered more than the speed of the processor. This section makes precise what "one operation" even means for runtime analysis. The short answer: we count **constant-time operations** – the unit of work the CPU can do in a fixed time independent of input values.

Definition — Constant-time operation

A **constant-time operation** is one whose execution time does not depend on the size of the input data. The same operation takes roughly the same amount of time whether the operand is small or large. We call this $O(1)$.

Key idea Why we count operations instead of seconds

Seconds change with the machine. Operations don't. If an algorithm does n constant-time operations, then on a $2\times$ faster CPU it takes half as long – but it still does n operations. Counting operations gives us a measure of an algorithm that is *about the algorithm*, not about the hardware.

8.1 What counts as constant time

The short list of things that are typically $O(1)$ on a modern processor:

Operation	Why it's constant
Arithmetic on fixed-size numeric types (`int`, `double`)	Hardware does it in a bounded number of cycles regardless of values.
Assignment of a fixed-size value (scalar, pointer, reference)	Copies a fixed number of bytes.
Comparison of fixed-size values	One CPU instruction.
Indexed access to an array element (`a[i]`)	One pointer arithmetic + one memory load.
Push / pop on a `std::vector` (amortized)	Constant on average; occasional resize is paid for across many pushes.
Hash-table lookup (*expected*)	One hash computation + one bucket probe on average.
Dereferencing a pointer	One memory load.
Reading the length stored by a container (`v.size()`)	Returns a cached value.

Example Operations that *look* simple but aren't constant time

```
std::string a(1'000'000, 'x');
std::string b(1'000'000, 'y');

std::string c = a + b;          // O(n) -- allocates and copies
std::string d = a;              // O(n) -- copy constructor
bool same = (a == b);           // O(n) -- compares every character
int count = std::count(v.begin(), v.end(), 3);  // O(n)
```

The +, =, and == operators look like one operation at the language level, but they walk every character of every string. The source code has constant width; the runtime does not.

Gotcha "It's one line" $\neq$ "it's $O(1)$"

The biggest beginner mistake in complexity analysis is confusing *syntax* with *semantics*. Any operation on a container whose size grows with input – concatenation, copying, containment checks, sort, reverse, clear of a large structure – has a cost proportional to that size. Even when the call site is a single token, the work isn't free.

8.2 Why fixed size matters

Notice the repeating phrase: "fixed-size integer or floating-point values." A modern CPU's adder has a fixed number of bits – 32 or 64 – and its time-per-add doesn't depend on whether the operands are 3 or 2,147,483,646.

Key idea Arbitrary-precision arithmetic is *not* $O(1)$

Python's built-in integers are arbitrary-precision: you can compute 10^{10000} directly. The trade-off is that arithmetic on them is no longer constant-time. Adding two n-digit numbers is $O(n)$; multiplying them is $O(n \log n)$ with fast algorithms, $O(n^2)$ with the schoolbook method.

C++ `int`, `long long`, and `double` are fixed-size by design. Arithmetic on them is genuinely constant-time – at the cost of overflow being possible. Different languages, same trade-off: you get either $O(1)$ arithmetic *or* unlimited precision, not both.

8.3 Counting the right things

In runtime analysis we don't count every line of code. We pick a representative operation and count how many times it runs.

Example What to count, by algorithm

- **Linear search.** Count *comparisons* between the search key and array elements. Each comparison is $O(1)$, so total work is proportional to the number of comparisons.
- **Sorting algorithms.** Count comparisons *and/or* swaps; both are constant-time.
- **Binary search.** Count iterations (or recursive calls); each iteration does constant work.

The hidden-loop trap

Gotcha Watch for hidden loops inside library calls

```
for (std::size_t i = 0; i < n; ++i) {
    std::string msg = "item " + std::to_string(i);  // O(len)
    output += msg;                                   // O(|output|)
}
```

This is **not** $O(n)$. The `output += msg` concatenation grows more expensive each iteration because `|output|` keeps growing. Total work is $O(n^2)$. A reader counting "one loop, so linear" gets the answer spectacularly wrong.

8.4 The rule of thumb

Key idea The "one at a time" heuristic

If an operation touches a *constant* amount of memory – one integer, one pointer, one cache line, one array cell – it's almost certainly $O(1)$. If it touches memory whose size grows with the input, assume it isn't. When in doubt:

1. Does it allocate?
2. Does it copy?
3. Does it call another function that loops?

Any "yes" is a hint that the operation is not constant-time.

Aside Where this is going

This section defined the unit of measurement. The next section (3.10, "Growth of functions and complexity") takes the count of constant-time operations – as a function of input size – and classifies algorithms by how that function grows. That classification is Big-O notation, formalized in the next section, §3.10.

9 3.10 Growth of Functions and Complexity

Section 3.9 defined the unit of measurement: constant-time operations. This section asks *how the count of those operations grows as the input grows.* That growth is what we summarize with Big-O (and its cousins Ω, Θ). These are the three core pieces of vocabulary that will structure every runtime claim for the rest of the course.

9.1 Best case, worst case, and $T(N)$

For an algorithm running on input of size N, the number of constant-time operations it performs is a function of N. We'll call that function $T(N)$. Usually $T(N)$ depends on more than just N – it depends on which particular input of size N we're running on. Hence the standard two-sided view:

Definition — Best / worst case runtime

Best-case $T(N)$ is the fewest constant-time operations the algorithm could do on any input of size N. **Worst-case** $T(N)$ is the most. Average-case is an in-between notion that requires a probability distribution over inputs.

Example Linear search as a two-sided picture

- Best case: key is at index 0, so $T_{\text{best}}(N) = 1$.
- Worst case: key is last, or absent, so $T_{\text{worst}}(N) = N$.

Both are true statements about the same algorithm. You report whichever matches the guarantee you're after.

9.2 Upper and lower bounds

Rather than carrying the exact $T(N)$ formula around (with all its constants and lower-order terms), we bound it with simpler functions.

Definition — Upper / lower bound

A function $f(N)$ is an **upper bound** on $T(N)$ if, for all sufficiently large N, $T(N) \leq f(N)$. It's a **lower bound** if $T(N) \geq f(N)$ for all sufficiently large N.

Example Bounds for a messy $T(N)$

Suppose an algorithm's worst-case runtime is

$$T(N) = 3N^2 + 10N + 17.$$

- $f(N) = 30N^2$ is an upper bound. ($30N^2 \geq 3N^2 + 10N + 17$ for all $N \geq 1$.)
- $f(N) = N^2$ is a lower bound.

Either bound gives a cleaner statement than the exact $T(N)$, at the cost of some precision.

9.3 Asymptotic notation: strip the constants

For cross-algorithm comparison we don't care about the constants in front of N^2. We care that it's an N^2. That's what Big-O, Ω, and Θ formalize: *rate of growth, constants and lower-order terms absorbed.*

Definition — Asymptotic notations

Let $T(N)$ be an algorithm's runtime (in constant-time operations) and $f(N)$ a simpler reference function. Then:

- $T(N) = O(f(N))$ means there's a positive constant c with $T(N) \leq c \cdot f(N)$ for all $N \geq N_0$. *Big-O is an upper bound on growth rate.*
- $T(N) = \Omega(f(N))$ means there's a positive constant c with $T(N) \geq c \cdot f(N)$ for all $N \geq N_0$. *Big-Ω is a lower bound on growth rate.*
- $T(N) = \Theta(f(N))$ means $T(N)$ is both $O(f(N))$ and $\Omega(f(N))$ – the two bounds meet. *Big-Θ is a tight bound.*

Key idea The mental picture

- O: "The runtime doesn't grow faster than this."
- Ω: "The runtime doesn't grow slower than this."
- Θ: "The runtime grows exactly like this."

O is what you say when you want to promise *it won't blow up*; Ω is what you say to argue *it can't be better than this*; Θ is what you say when you've pinned it from both sides.

Aside Little-oh and little-omega: when the bound is strictly looser

O and Ω may or may not be *tight* – $2n = O(n^2)$ is true (linear grows no faster than quadratic), but it's also a loose bound. CLRS introduces two more notations for the strictly-looser case:

- $f(n) = o(g(n))$ (**little-oh**): for *every* positive constant c, there exists n_0 such that $f(n) < c \cdot g(n)$ for all $n \geq n_0$. Equivalently, $\lim_{n\to\infty} f(n)/g(n) = 0$. Strict upper bound.
- $f(n) = \omega(g(n))$ (**little-omega**): the mirror image. $\lim_{n\to\infty} f(n)/g(n) = \infty$. Strict lower bound.

The cleanest mental model is the analogy with real-number comparisons:

$$\begin{array}{lll} f = O(g) & \text{like } f \leq g & \\ f = \Omega(g) & \text{like } f \geq g & \\ f = \Theta(g) & \text{like } f = g & \\ f = o(g) & \text{like } f < g & \text{(strict)} \\ f = \omega(g) & \text{like } f > g & \text{(strict)} \end{array}$$

So $2n = O(n^2)$ AND $2n = o(n^2)$ – the latter is the precise statement (linear is *strictly* smaller than quadratic). $2n = O(n)$ is also true, but $2n \neq o(n)$ – linear is not strictly smaller than itself. Use o and ω when you want to signal "definitely not just a loose bound."

What c and N_0 buy you

The constant c absorbs the hardware, the language, and any constant-factor overhead. The threshold N_0 lets us *ignore small inputs*, where lower-order terms and one-time setup costs distort the picture.

Example Why the constants go away

Let $T(N) = 3N^2 + 10N + 17$. Then $T(N) = O(N^2)$ because for $c = 30$ and $N \geq 1$,

$$3N^2 + 10N + 17 \;\leq\; 3N^2 + 10N^2 + 17N^2 \;=\; 30N^2.$$

We gave up the exact coefficient (3) to get a clean, hardware-free statement: this algorithm's runtime *grows like* N^2.

9.4 How to read a big-O claim

Gotcha Common traps

- **Big-O is *not* the "worst case."** You can have Big-O of *any* case – best, worst, average. "Linear search worst case is $O(N)$" is a bound on the worst case. "Linear search best case is $O(1)$" is a bound on the best case. Both are legitimate.
- **Big-O doesn't promise tightness.** Saying "linear search worst case is $O(N^2)$" is technically true – N grows no faster than N^2. It's just a loose upper bound. That's why we prefer Θ when we can prove it.
- **Constants and lower-order terms are not mysteries.** They're real; $100N$ is slower than N in practice. Big-O is a coarse tool for comparing *shapes*, not a license to ignore the constant when performance matters.

9.5 The growth-rate hierarchy

For large N, the following functions are ordered by how fast they grow. Every entry dominates the ones above it.

Class	Name	Examples
$O(1)$	Constant	Array index, hash lookup
$O(\log N)$	Logarithmic	Binary search, balanced-BST lookup
$O(N)$	Linear	Linear search, one pass over an array
$O(N \log N)$	Linearithmic / "$N \log N$"	Merge sort, heap sort, good sorts
$O(N^2)$	Quadratic	Bubble sort, insertion sort worst case, nested loops
$O(N^3)$	Cubic	Naive matrix multiply, certain DP algorithms
$O(2^N)$	Exponential	Brute-force subset enumeration
$O(N!)$	Factorial	Brute-force permutations (traveling salesman)

Key idea The gaps between classes are enormous

At $N = 100$ (only!), the difference between the classes is already wild:

Class	ops at $N = 100$
$O(1)$	1
$O(\log N)$	7
$O(N)$	100
$O(N \log N)$	≈ 664
$O(N^2)$	10,000
$O(2^N)$	$\approx 1.27 \times 10^{30}$
$O(N!)$	$\approx 9.33 \times 10^{157}$

The jump from quadratic to exponential is where problems go from "slow" to "literally cannot be solved in the lifetime of the universe for modest N."

9.6 In practice

Aside What you'll actually say in code reviews

In day-to-day work you almost always state runtime as Big-O of the worst case: "`std::unordered_map::find` is $O(1)$ expected," "your nested loop is $O(N^2)$," "this sort is $O(N \log N)$." Ω and Θ show up in analysis problems and lower-bound arguments more than in casual engineering chatter – but knowing them lets you read correct statements like "no comparison sort can do better than $\Omega(N \log N)$."

Key idea Halfway recap – before we dive into sorts

You've now been through the first half of ch. 3. Before sorting algorithms start piling up, consolidate:

What you've built (3.1-3.10):

1. **Bits & ASCII.** Data in memory is bits; characters are just small integers with an agreed table. You can move between `char`, `int`, and `cctype` classification without friction.
2. **Seven data structures named.** Array, linked list, tree, BST, heap, hash table, graph. You can't implement them all yet, but you know their names and what shape of problem each is good for.
3. **ADT vs. data structure.** An ADT is the *contract* ("a stack has push/pop/top" — no mention of *how*). A data structure is the *implementation*. Every C++ STL container is an ADT realized by one or more data structures (`std::stack<T>` is an ADT; its default implementation is `std::deque<T>`).
4. **Two searches contrasted.** Linear search is $O(n)$ and works on any data. Binary search is $O(\log n)$ but requires sorted data — which motivates the next half of the chapter.
5. **Big-O as a working tool.** You can read "constant time," "logarithmic," "linear," "quadratic," "exponential" and point at the growth-rate hierarchy.

The through-line into sorting. Binary search is the payoff of sorted data. That makes "how expensive is sorting?" the immediate follow-up question: if sorting costs $O(n \log n)$ and then you do one search at $O(\log n)$, you've paid more than linear search ($O(n)$) for one lookup — so sorting only pays when you search *many* times. Every sort in the next half of the chapter is a different attempt to make that one-time cost as small as possible.

What's ahead (3.14-3.21): seven sorting algorithms plus a fast-sort overview. They split into three families:

- **Incremental:** bubble (covered in ch_13 extras as pedagogy-only), insertion, selection, shell — easy to code, $O(n^2)$ (except shell).
- **Divide-and-conquer:** merge sort, quicksort — $O(n \log n)$ average; the ones you'll actually ship.
- **Non-comparison:** radix sort — $O(nk)$, fast when k (key width) is small.

If any of 3.1-3.10 still feels shaky, stop here and re-read. The sorting analyses that follow all lean on the Big-O definition and the growth-rate hierarchy. You don't need mastery yet; you need enough fluency that "this loop is $O(n)$" and "this outer-over-inner loop is $O(n^2)$" don't slow you down.

10 3.14 Sorting: Introduction

Searching (3.6, 3.8) was the first algorithmic problem we analyzed. **Sorting** is the second, and it's arguably the more important one: binary search, priority queues, deduplication, many-pass merges, and much of

computational geometry all assume data you can sort first. This short section sets the stage; the rest of this chapter walks through insertion, selection, shell, quicksort, merge, and radix sort. (Bubble sort lives in ch_13's extras chapter, since it is pedagogy-only; heap sort waits for ch_7, once we have heaps.)

Definition — Sorting

Sorting is the process of rearranging a sequence of elements into non-decreasing ("ascending") or non-increasing ("descending") order under some total ordering. For numbers, the order is $\leq$; for strings, usually lexicographic; for custom types, whatever comparison function you supply.

10.1 Why sorting is its own field

Sorting is the most-studied algorithmic problem in computer science. It looks trivial at small sizes – you could do it by hand – and turns out to be surprisingly deep.

Key idea What sorting unlocks

Sorted data is cheaper to work with in several important ways.

- **Binary search.** $O(\log n)$ lookup instead of $O(n)$.
- **Duplicates.** Duplicates end up adjacent – a single pass detects or removes them.
- **Set operations.** Intersection, union, and difference of two sorted sequences are each $O(n + m)$ (merge-style).
- **k-th smallest / largest.** Once sorted, it's an index lookup.
- **Human consumption.** Sorted output is the default contract of almost every report, table, and leaderboard.

Much of the time, the cost of sorting ($O(n \log n)$) is amortized across many subsequent queries that are each much cheaper because of the sort.

10.2 The fundamental constraint

Why isn't sorting just "put each element in its right slot"? Because the program *doesn't know* where the right slot is without doing work to find out.

Definition — The access constraint

A sorting algorithm can't see the whole list at once. It can only *observe or swap a small number of elements at a time* – typically two, via a comparison and a conditional swap. Sorting algorithms are the art of doing this efficiently.

Example Swap as the atomic move

```
void swap(int& a, int& b) {
    int tmp = a;
    a = b;
    b = tmp;
}

// Or just:
std::swap(a, b);
```

Almost every comparison-based sort boils down to compare-two, swap-if-needed, repeated in different patterns. Bubble sort scans adjacent pairs. Selection sort finds the minimum of a suffix and swaps it into place. Insertion sort shifts a value leftward through an already-sorted prefix. Quicksort partitions around a pivot. Merge sort combines sorted halves.

Aside Why we can't "just place each element in its slot"

Placing each element in its final slot would mean *already knowing* where it goes, which means already having done the sort. An algorithm starts knowing nothing beyond the ability to compare two elements and swap two slots. The creativity in sorting algorithms is in deciding which comparisons to make and in what order.

10.3 What makes sorts differ

When we compare two sorting algorithms we ask five questions:

Property	What it means
Time complexity	Best / average / worst-case count of comparisons and swaps as a function of n.
Space complexity	Extra memory beyond the input. *In-place* sorts use $O(1)$ extra; merge sort needs $O(n)$.
Stability	Does the sort preserve the original order of *equal* elements? Useful when you've already sorted by a secondary key.
Adaptivity	Does it run faster on already-mostly-sorted input?
Access pattern	Does it need random access (quicksort, binary search during sort) or sequential only (merge sort)?

Key idea The big-picture result

There's a proven lower bound: *any comparison-based sort does $\Omega(n \log n)$ comparisons in the worst case.* Merge sort, heap sort, and well-implemented quicksort all meet this bound asymptotically. Bubble sort, selection sort, and insertion sort do not – they're $\Theta(n^2)$. That gap is why "toy sorts" are pedagogy and "real sorts" are engineering.

Faster-than-comparison sorts

Aside Specialized sorts can beat $n \log n$

If you have more information about the values (say, they're small non-negative integers, or short fixed-length keys), *non-comparison* sorts – counting sort, radix sort, bucket sort – can run in $O(n)$ or $O(n + k)$. The $\Omega(n \log n)$ lower bound only applies when the algorithm's only tool is comparison.

10.4 In C++

Example The library already sorts

```
#include <algorithm>
std::vector<int> v = {17, 3, 44, 6, 9};
std::sort(v.begin(), v.end());                      // ascending: 3 6 9 17
    44
std::sort(v.begin(), v.end(), std::greater<int>()); // descending

std::stable_sort(v.begin(), v.end());               // preserves equal-
    order
std::partial_sort(v.begin(), v.begin() + 3, v.end()); // only first 3
    sorted
```

`std::sort` is typically an *introsort*: quicksort that switches to heap sort when recursion gets too deep. Worst-case $O(n \log n)$, average $O(n \log n)$ with a tiny constant. For production code, start here.

Key idea Where this is going

The rest of the sorting material walks through the classic algorithms: bubble, selection, and insertion sort (simple, quadratic, pedagogical); then quicksort, merge sort, and heap sort ($n \log n$, practical). Each is a different answer to the same question: "given only compare and swap, in what order should I issue them?" Understanding why the fast sorts are fast is the main payoff of the complexity chapter.

11 3.15 Insertion Sort

Insertion sort is the first concrete sorting algorithm we'll build. It's the sort most people *naturally* perform by hand – think about how you'd organize a hand of playing cards – and its quadratic worst-case cost is the price of its simplicity. It's also surprisingly good when the input is already nearly sorted, and that adaptivity is where it shines.

11.1 The idea

Definition — Insertion sort

Insertion sort partitions the array into a *sorted prefix* and an *unsorted suffix*. On each step, it takes the next element from the suffix and slides it leftward through the prefix until it's in the right spot, extending the sorted region by one.

Key idea Card-sorting analogy

Imagine picking up cards one at a time. The cards in your hand are always sorted; each new card from the deck gets walked leftward until it slots into place. Insertion sort is exactly that, in code.

11.2 Insertion sort in C++

Example Swap-based insertion sort

```
void insertionSort(std::vector<int>& nums) {
    for (std::size_t i = 1; i < nums.size(); ++i) {
        std::size_t j = i;
        while (j > 0 && nums[j] < nums[j - 1]) {
            std::swap(nums[j], nums[j - 1]);
            --j;
        }
    }
}
```

The outer loop marks the boundary between sorted and unsorted. The inner loop walks `nums[i]` leftward one swap at a time until its left neighbor is no larger.

Example Shift-based insertion sort (one fewer write per step)

```
void insertionSort(std::vector<int>& nums) {
    for (std::size_t i = 1; i < nums.size(); ++i) {
        int key = nums[i];
        std::size_t j = i;
        while (j > 0 && nums[j - 1] > key) {
            nums[j] = nums[j - 1];   // shift right
            --j;
        }
        nums[j] = key;               // drop key into place
    }
}
```

Instead of swapping pairs (three writes each), this version shifts and then drops the key once. Same algorithm, measurably faster constant factor.

Aside Two versions, same algorithm

Swap-based is the textbook version – it reads more naturally. The shift-based version is the one you'd write after profiling. Either is $O(n^2)$ in the worst case; the shift-based version has a smaller constant.

11.3 Worst-case analysis: $O(n^2)$

Key idea Counting comparisons

The outer loop runs $n-1$ times. On iteration i, the inner loop does at most i comparisons – it might walk the value all the way to index 0. Summing:

$$1+2+3+\cdots+(n-1)=\frac{n(n-1)}{2}\in\Theta(n^2).$$

This maximum is achieved when the input is in *reverse sorted order* – every new element has to travel all the way to the front.

11.4 Best case: $O(n)$ on already-sorted input

Key idea Why insertion sort is *adaptive*

If the input is already sorted, the inner while condition `nums[j]<nums[j - 1]` is false on the very first check of every iteration. The inner loop does one comparison and terminates. Total: $n-1$ comparisons, zero swaps – $\Theta(n)$.

Nearly sorted lists

The adaptivity generalizes. Suppose only a constant number c of elements are out of place. Then

- $n-c$ elements each need 1 comparison to stay put.
- c elements each need at most n comparisons.

Total work: $(n-c)\cdot 1+c\cdot n=n-c+cn\in O(n)$ (since c is constant). Insertion sort runs in *linear time* on nearly-sorted input.

Example Which lists are "nearly sorted"?

$(1,2,3,4,5,6)$	Sorted
$(1,2,3,4,5,10,6)$	Nearly sorted (one misplaced)
$(4,5,17,25,89,14)$	Nearly sorted
$(6,5,4,3,2,1)$	Worst case – fully reversed
$(9,3,7,1,8,2)$	Random – expect $\Theta(n^2)$

11.5 Why this matters in practice

Key idea Insertion sort's real-world niche

Given its $O(n^2)$ worst case, insertion sort sounds like a pedagogical stop on the way to quicksort. In practice it's still used, in two specific places:

- **Small inputs.** For n below roughly 10–32, the tiny constant factor beats the overhead of fancier sorts. Introsort (`std::sort`) actually switches to insertion sort at the leaves of its recursion for exactly this reason.
- **Streaming input.** If elements arrive one at a time and you want the collection kept sorted, insertion sort is what you're doing whether you named it or not.

11.6 Properties

Property	Insertion sort
Best case	$\Theta(n)$ – already-sorted input
Worst case	$\Theta(n^2)$ – reverse-sorted input
Average case	$\Theta(n^2)$ for random input
Space	$O(1)$ extra – in-place
Stable?	Yes – equal elements retain original order
Adaptive?	Yes – fast on nearly-sorted input

Gotcha Stability needs the strict inequality

The inner test is `nums[j] < nums[j - 1]`, *not* <=. Using <= would swap equal elements, which destroys stability. When you're sorting by a secondary key on top of a previous sort, stability matters – use strict <.

Aside Loop invariants: the formal correctness-proof technique

The intuitive argument "insertion sort works because each pass extends the sorted prefix by one" is a *loop invariant* proof, just stated informally. CLRS Ch 2.1 makes this rigorous – the same 3-part structure works for every loop-based algorithm in this course (binary search, BST insert, Dijkstra, ...).

The technique. For insertion sort's outer loop, the invariant is:

> *At the start of each iteration, the subarray* `nums[0..i-1]` *contains the original first i elements, sorted in non-decreasing order.*

A complete proof has three parts:

1. **Initialization.** The invariant holds before the first iteration. Here: `i = 1`, so the prefix is `nums[0..0]` – one element, trivially sorted.
2. **Maintenance.** If the invariant holds at the start of one iteration, it holds at the start of the next. Here: the inner while loop slides `nums[i]` leftward past anything larger; when it stops, the prefix `nums[0..i]` is sorted. Increment `i`; invariant restored for next iteration.
3. **Termination.** When the loop exits, the invariant plus the exit condition imply the desired result. Here: `i == nums.size()`, so the entire array is the sorted prefix.

That's a complete correctness argument. The same scaffold scales to harder algorithms – binary search's invariant is "the target, if present, lies in `[lo, hi]`"; BST search's is "the target, if present, lies in the subtree rooted at the current node." Once you can state the invariant, the proof writes itself.

Aside Where this is going

The next sorting sections introduce quicksort (fast average, worst-case n^2 if unlucky with pivots), merge sort (guaranteed $n \log n$, $O(n)$ space), and heap sort (guaranteed $n \log n$, in-place). Each earns its place in the toolbox by trading off among the five properties in the table above.

12 3.16 Selection Sort

Selection sort is insertion sort's mirror image. Both carve the array into a sorted prefix and an unsorted suffix; both extend the prefix by one element per outer-loop pass. The difference is *which element* gets placed each pass. Insertion sort takes whatever is next and slides it into position. Selection sort picks the *smallest element of the remaining unsorted region* and parks it at the end of the sorted region.

Definition — Selection sort

Selection sort partitions the array into a sorted prefix and an unsorted suffix. On each pass, it scans the suffix to *select* the smallest (or largest, for descending) remaining element and *swaps it* into the first position of the suffix, extending the sorted prefix.

12.1 Selection sort in C++

Example Selection sort

```
void selectionSort(std::vector<int>& nums) {
    for (std::size_t i = 0; i + 1 < nums.size(); ++i) {
        std::size_t indexSmallest = i;
        for (std::size_t j = i + 1; j < nums.size(); ++j) {
            if (nums[j] < nums[indexSmallest]) {
                indexSmallest = j;
            }
        }
        std::swap(nums[i], nums[indexSmallest]);
    }
}
```

Outer loop advances the boundary. Inner loop scans the suffix and records the index of the smallest. One swap per outer iteration places it.

Gotcha Avoid `i < size - 1` on unsigned types

`std::vector::size()` returns `size_t` (unsigned). If the vector is empty, `size - 1` underflows to a huge number and the loop runs forever. Write `i + 1 < nums.size()` instead – it's the same condition for non-empty input and safe for empty input.

12.2 Why it always does $\Theta(n^2)$ comparisons

Key idea Counting comparisons

On pass i (zero-based), the inner loop compares every suffix element against the current smallest – that's $n - i - 1$ comparisons. Summing:

$$(n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2} \in \Theta(n^2).$$

The key detail is that the number of comparisons *does not depend on the data.* Even if the array is already sorted, the inner loop still scans the entire suffix to *prove* it has found the minimum. Every case – best, average, worst – is $\Theta(n^2)$.

Gotcha Selection sort is not adaptive

Insertion sort runs in $O(n)$ on nearly-sorted input; selection sort does not. It does the same work no matter what. If your input is already mostly sorted, reach for insertion sort instead.

12.3 Where selection sort *is* better than insertion sort

Selection sort has one redeeming feature: it does at most $n - 1$ *swaps* total. Insertion sort can do $\Theta(n^2)$ swaps.

Key idea When swap cost dominates

If a "swap" is expensive – say each element is a large struct that's costly to move, or is stored in flash memory where writes wear out the hardware – selection sort wins. It finds the right element via cheap comparisons, then does one swap per pass. You trade comparison work for write work, and sometimes the writes are what you're actually trying to minimize.

Selection sort vs. insertion sort at a glance

Property	Insertion sort	Selection sort
Best case	$\Theta(n)$	$\Theta(n^2)$
Worst case	$\Theta(n^2)$	$\Theta(n^2)$
Space	$O(1)$	$O(1)$
Stable?	Yes	No (swap can reorder equals)
Adaptive?	Yes	No
Swaps	Up to $\Theta(n^2)$	Exactly $n - 1$

Gotcha Selection sort is *not* stable – a subtle why

Consider sorting $[5_a, 3, 5_b, 1]$ (the subscripts distinguish two "5"s with equal key but different identity). Pass 0 finds the smallest (1) at index 3 and swaps it with index 0, giving $[1, 3, 5_b, 5_a]$. The order of the two 5's has flipped even though they're equal by key. Any secondary ordering you had *is gone*. That's why when stability matters – e.g. sorting rows by column A after already sorting by column B – use insertion sort or merge sort, not selection sort.

12.4 When to reach for each

Key idea A pocket rubric

- **Insertion sort.** Small n, streaming data, nearly sorted input, or when stability matters.
- **Selection sort.** Small n where *writes* are much more expensive than comparisons.
- **Otherwise.** `std::sort` (introsort).

For real work, the third bullet covers 99% of cases. The first two are about understanding the trade-offs so you know *why* the third bullet is the default.

Aside The pedagogical point

Insertion and selection sort together illustrate that "$O(n^2)$" isn't one algorithm – it's a class, with members that differ in *which* operation they do n^2 of (comparisons vs swaps), whether they adapt to input shape, and whether they preserve equal-element order. Analyzing any new sort means asking the same checklist.

13 3.17 Shell Sort

Shell sort is the first sort we meet that breaks out of the quadratic class – not by doing something dramatically new, but by applying insertion sort cleverly in *multiple passes over interleaved sublists*. It's the bridge between the simple sorts of 3.15–3.16 and the $O(n \log n)$ sorts of the next chapter. Named for Donald Shell, 1959.

13.1 The core insight

Insertion sort is slow on unsorted input because each misplaced element can only move *one slot at a time*. A tiny value at the end of a big array has to be swapped past every larger neighbor, one pair at a time, to reach the front – that's the n^2.

Key idea Move elements in big jumps first

What if elements could *leap* most of the distance in a few moves, and only the last little bit was done by ordinary insertion sort (which is $O(n)$ on nearly-sorted input)? That's Shell sort: first run insertion sort on many interleaved subsequences with a big "gap," which lets elements jump far; then shrink the gap repeatedly; finish with gap = 1 (ordinary insertion sort) on a list that's already almost sorted.

13.2 Interleaved lists and the "gap"

Definition — Gap

A **gap** K is a positive integer. With gap K, Shell sort treats the array as K *interleaved* sublists: indices $0, K, 2K, \ldots$; then $1, K+1, 2K+1, \ldots$; and so on through index $K-1$. Each sublist is sorted by an insertion-sort variant that uses K as the step size instead of 1.

Example Gap of 3 on a 9-element list

Array: $[56, 42, 93, 12, 77, 82, 75, 91, 36]$
With gap 3, the three interleaved sublists are:

- indices $\{0, 3, 6\}$: $(56, 12, 75)$
- indices $\{1, 4, 7\}$: $(42, 77, 91)$
- indices $\{2, 5, 8\}$: $(93, 82, 36)$

Each is sorted independently. After sorting the three interleaved lists, the array looks like: $[12, 42, 36, 56, 77, 82, 75, 91, 93]$.
The array isn't sorted yet, but it's *close*: the three smallest elements all ended up in the first three slots. That's the setup Shell sort creates so that the final gap-1 pass (plain insertion sort) runs in nearly linear time.

13.3 Shell sort in C++

Example Shell sort implementation

```
// Insertion sort with a custom step size
static void insertionSortGap(std::vector<int>& nums,
                             std::size_t start,
                             std::size_t gap) {
    for (std::size_t i = start + gap; i < nums.size(); i += gap) {
```

```
        std::size_t j = i;
        while (j >= gap && nums[j] < nums[j - gap]) {
            std::swap(nums[j], nums[j - gap]);
            j -= gap;
        }
    }
}

void shellSort(std::vector<int>& nums) {
    for (std::size_t gap = nums.size() / 2; gap > 0; gap /= 2) {
        for (std::size_t start = 0; start < gap; ++start) {
            insertionSortGap(nums, start, gap);
        }
    }
}
```

The outer loop shrinks the gap (halving here). For each gap, the inner loop runs an insertion-sort variant on each of the K interleaved sublists. The final pass uses gap = 1, which is plain insertion sort.

Gotcha You *must* end with a gap of 1

A Shell sort that never runs with gap = 1 doesn't actually sort. It just produces an array whose interleaved sublists are sorted. The final gap = 1 pass is what finishes the job.

13.4 Choice of gap sequence

Definition — Gap sequence

A **gap sequence** is the descending list of gap values Shell sort uses, ending with 1. Any sequence works *for correctness*, but the sequence dramatically affects the *runtime*.

Gap sequence	Example ($n = 100$)	Worst-case runtime
Halving: $n/2, n/4, \ldots, 1$ (original Shell)	50, 25, 12, 6, 3, 1	$O(n^2)$
Hibbard: $2^k - 1$	63, 31, 15, 7, 3, 1	$O(n^{3/2})$
Sedgewick / Knuth	121, 40, 13, 4, 1	$O(n^{4/3})$
Ciura (empirical, best known small-n)	57, 23, 10, 4, 1	very fast in practice

Key idea The strange fact about Shell sort

Shell sort's exact asymptotic worst case is still open for some gap sequences. For Hibbard ($2^k - 1$) it's $O(n^{3/2})$. For Sedgewick's sequence it's $O(n^{4/3})$. Ciura's sequence isn't known in closed form but beats the others in benchmarks. Shell sort is one of the few algorithms in everyday use whose worst-case complexity depends on an unproven conjecture about a numeric sequence.

13.5 Why this beats plain insertion sort

Key idea Why the gap matters

Plain insertion sort moves misplaced elements *one slot per swap.* If an element is d slots from its destination, it needs d swaps. Summed over all elements, that's the n^2.
With gap K, a swap moves an element K slots. A few gap-K passes can reduce the *total distance every element has to travel* dramatically before the final gap-1 cleanup. The cleanup is still insertion sort, but on a list where elements are at most a small constant away from their destination – and insertion sort on nearly-sorted data is $O(n)$.

13.6 Properties

Property	Shell sort
Best case	$O(n \log n)$ (already sorted, depends on sequence)
Worst case	$O(n^2)$ (naïve halving) down to $O(n^{4/3})$ (Sedgewick)
Average case	Depends on gap sequence; between $O(n^{5/4})$ and $O(n^{3/2})$ in practice
Space	$O(1)$ extra – in-place
Stable?	No
Adaptive?	Somewhat – fewer passes on presorted input

Gotcha Shell sort is *not* stable

When gap > 1, swapping non-adjacent elements can reorder equal keys unpredictably. Use merge sort or insertion sort when stability is needed.

13.7 When to use it

Key idea Shell sort's niche

Shell sort is rarely the right choice in modern C++ code – `std::sort` beats it. But it has a few real uses:

- **Embedded / constrained environments.** In-place, simple to implement, no recursion stack, no extra memory. Useful when you can't afford $\log n$ stack depth.
- **Educational.** It's the easiest example of "clever insertion sort" and bridges n^2 and $n \log n$.
- **Moderate n in code without `<algorithm>`.** If you can't lean on the STL, Shell sort is a few lines of code that outperform plain insertion sort by a lot.

Aside Where this is going

Shell sort showed that *structure in the swap pattern* can break the n^2 barrier. The next step is algorithms that break it on purpose, with provable $O(n \log n)$ guarantees: quicksort, merge sort, and heap sort. Shell sort is the last sort where the question "what gap sequence do I use?" really matters – after this, the algorithms have more definite shapes.

14 3.18 Quicksort

Quicksort is the first algorithm we meet that achieves $O(n \log n)$ *on average* – and it's the most widely used comparison sort in practice. Its strategy is a textbook example of **divide-and-conquer**: do a bit of work to partition the problem into two smaller subproblems, recurse on each, and stitch the result together. Invented by Tony Hoare in 1959 while he was a graduate student trying to sort Russian words for machine translation.

14.1 The idea: partition and recurse

Definition — Quicksort

Quicksort sorts a range by:

1. Choosing a **pivot** value from the range.
2. **Partitioning** the range so every value $\leq$ pivot is on the left, every value $\geq$ pivot is on the right.
3. Recursively quicksorting the left side and the right side.

The range is sorted when both recursive calls return.

Key idea Why partitioning works

Once partitioning is done, *every element on the left is $\leq$ every element on the right.* So sorting the two sides independently sorts the whole range – no merge step needed. The work to combine the results is zero. That's what makes quicksort an in-place sort.

14.2 The Hoare-style partition

The textbook's partition is a variant of Hoare's original. Two index pointers walk from the ends toward each other, swapping out-of-place values when they meet misplaced elements.

Example Hoare partition in C++

```
static std::size_t partition(std::vector<int>& nums,
                             std::size_t low,
                             std::size_t high) {
    std::size_t mid = low + (high - low) / 2;
    int pivot = nums[mid];

    while (true) {
        while (nums[low]  < pivot) ++low;
        while (pivot < nums[high]) --high;

        if (low >= high) return high;

        std::swap(nums[low], nums[high]);
        ++low;
        --high;
    }
}

void quicksort(std::vector<int>& nums,
```

```
                std::size_t low,
                std::size_t high) {
    if (low >= high) return;                          // base case
    std::size_t mid = partition(nums, low, high);
    quicksort(nums, low, mid);                         // left half
    quicksort(nums, mid + 1, high);                    // right half
}
```

Pivot picked at the midpoint; two pointers walk inward; swap when both find something on the wrong side; stop when they meet. After the partition returns, each side is recursively sorted.

Gotcha Watch the pivot-index caveat

Hoare's partition returns an index j such that everything in $[low, j]$ is $\leq$ pivot and everything in $[j+1, high]$ is $\geq$ pivot. The *pivot value itself might end up on either side*, and there's no guarantee the pivot sits at index j. That's why the recursive calls use `[low, mid]` and `[mid + 1, high]` – never excluding the middle index, as you'd do in Lomuto's partition. Getting this wrong is a classic bug.

Aside Lomuto vs. Hoare partition

Two partition schemes are standard. **Hoare** (shown above) uses two converging pointers; it's faster in practice but *doesn't* place the pivot at its final index. **Lomuto** uses a single running "boundary" index and pivots around the *last* element; it's easier to get right, slower by a constant, and places the pivot at its final sorted index. Intro CS courses often teach Lomuto; introsort (used in `std::sort`) uses a Hoare variant.

14.3 Runtime: average vs. worst

Key idea Average case: $O(n \log n)$

If the pivot splits the range into two roughly equal halves, the recursion tree has depth $\log_2 n$ and at each level quicksort does $\Theta(n)$ work partitioning. Total: $\Theta(n \log n)$.

$$\text{levels} \times \text{work per level} \;=\; \log n \;\times\; n \;=\; n \log n.$$

This is what happens on random or "normal" data.

Gotcha Worst case: $O(n^2)$

If the pivot is always the smallest (or largest) element, one side gets zero elements and the other gets $n-1$. The recursion depth becomes n, and total work is $\Theta(n^2)$:

$$n + (n-1) + (n-2) + \cdots + 1 = \frac{n(n+1)}{2}.$$

This is especially bad for already-sorted or reverse-sorted input if the pivot is "first" or "last." Naïve quicksort on sorted data is actually *slower than insertion sort.*
Picking the middle element as pivot (as the textbook does) partially mitigates this – already-sorted inputs partition cleanly. Adversarial inputs can still force $O(n^2)$, which is why real implementations use more defensive strategies.

14.4 Pivot selection strategies

Strategy	Behavior
First or last element	Disastrously $O(n^2)$ on sorted inputs. Don't.
Middle element	Good for sorted inputs; still $O(n^2)$ on adversarial data.
Random element	Expected $O(n \log n)$ no matter the input pattern. Adversary can't predict the pivot.
Median-of-three (first, middle, last)	Cheap approximation of the true median; very robust in practice. The choice most real implementations make.
Introsort	Start with quicksort, but if recursion depth exceeds $2 \log n$, switch to heap sort. Guaranteed $O(n \log n)$ worst case. `std::sort` does this.

Key idea Why `std::sort` is basically always good enough

`std::sort` uses introsort:

- Quicksort with median-of-three pivot selection,
- Switches to heap sort if recursion is getting too deep (guarding the worst case),
- Switches to insertion sort on small subranges (where insertion sort's small constant wins).

You get average quicksort speed, worst-case heap-sort guarantee, and small-range insertion-sort wins, all in one call.

14.5 Properties

Property	Quicksort
Best case	$\Theta(n \log n)$
Average case	$\Theta(n \log n)$
Worst case	$\Theta(n^2)$ – mitigated by pivot strategy
Space	$O(\log n)$ recursion stack (in-place for the data)
Stable?	No – partition swaps non-adjacent elements
Adaptive?	Not especially – depends on pivot choice

Gotcha Not stable, again

If stability matters – you're sorting by a secondary key on top of a previous sort – use `std::stable_sort` (merge sort) or insertion sort, not `std::sort`.

14.6 Why is it called *quick* if it can be $O(n^2)$?

Key idea The constant factor is extremely small

On random data, quicksort does noticeably fewer operations per element than merge sort or heap sort, and its inner loop is very cache-friendly: it streams through memory sequentially and only touches two array slots at a time. "Quick" refers to real wall-clock performance, not asymptotic analysis. Modern engineered quicksort (introsort) beats both merge sort and heap sort on typical inputs, and avoids the $O(n^2)$ pathology by design.

Aside Where this is going

Quicksort is average-case optimal and the default sort in every major language. The next algorithm, merge sort, gives up the in-place property in exchange for a guaranteed $O(n \log n)$ worst case and stability – useful when you cannot tolerate the rare quadratic blowup or when the data is in a linked list (where partitioning doesn't work but merging does).

15 3.19 Merge Sort

Quicksort gets its $O(n \log n)$ *on average* and pays a rare $O(n^2)$ on bad pivots. Merge sort buys a *guaranteed* $O(n \log n)$ worst case plus stability, at the cost of $O(n)$ extra memory. It's also divide-and-conquer, but with the work distributed differently: quicksort does the work *before* recursing (partition then recurse), merge sort does the work *after* (recurse then merge). Invented by John von Neumann in 1945.

15.1 The idea: recurse then merge

Definition — Merge sort

Merge sort sorts a range by:

1. If the range has ≤ 1 element, it's already sorted. Return.
2. **Divide** the range into two halves at the midpoint.
3. Recursively merge-sort each half.
4. **Merge** the two sorted halves into a single sorted range.

Key idea The two halves of divide-and-conquer

Quicksort does a smart partition (the hard work) and then the recursive calls trivially concatenate. **Merge sort** divides trivially (just split at the midpoint) and then does the hard work merging two already-sorted halves. The total work is the same, $\Theta(n \log n)$, but it lives in different places.

15.2 The merge step

The merge is where everything happens. Two already-sorted subsequences can be combined into one sorted sequence in linear time by walking both with two pointers and always taking the smaller of the two front elements.

Example Merging two sorted runs in C++

```
static void merge(std::vector<int>& nums,
                  std::size_t i, std::size_t j, std::size_t k) {
    // nums[i..j]   is the left  (sorted) half
    // nums[j+1..k] is the right (sorted) half
    std::vector<int> merged;
    merged.reserve(k - i + 1);

    std::size_t left = i, right = j + 1;
    while (left <= j && right <= k) {
        if (nums[left] <= nums[right]) {
            merged.push_back(nums[left++]);
```

```
        } else {
            merged.push_back(nums[right++]);
        }
    }
    while (left  <= j) merged.push_back(nums[left++]);
    while (right <= k) merged.push_back(nums[right++]);

    for (std::size_t p = 0; p < merged.size(); ++p) {
        nums[i + p] = merged[p];
    }
}
```

Scan both halves in lockstep. Whichever front element is smaller, take it. When one half is exhausted, flush the other. Finally, copy the merged buffer back to `nums[i..k]`.

Gotcha Stability lives in the <=

The test is `nums[left] <= nums[right]` ("if tied, take from the *left*"). This preserves the original order of equal elements – that's why merge sort is **stable**. If you change it to <, equals from the right get placed first and stability is lost.

15.3 Merge sort in C++

Example The recursive driver

```
void mergeSort(std::vector<int>& nums,
               std::size_t i, std::size_t k) {
    if (i >= k) return;                          // base case
    std::size_t j = i + (k - i) / 2;             // midpoint
    mergeSort(nums, i, j);                        // sort left
    mergeSort(nums, j + 1, k);                    // sort right
    merge(nums, i, j, k);                         // combine
}
```

The recursion tree has depth $\log_2 n$; each level does $\Theta(n)$ work in its merge calls. Total: $\Theta(n \log n)$.

15.4 Runtime: why always $O(n \log n)$

Key idea The recursion tree argument

Each level of the recursion:

- Splits all ranges at their midpoints – free, just an index calculation.
- Merges pairs of sorted subranges back together – each element of the original array is touched exactly once by a merge at that level, so total work per level is $\Theta(n)$.

The tree has depth $\lceil \log_2 n \rceil$. Total work:

$$\underbrace{\log_2 n}_{\text{depth}} \times \underbrace{n}_{\text{work per level}} = \Theta(n \log n).$$

Unlike quicksort, this bound is independent of input order. Best case, average case, and worst case are all $\Theta(n \log n)$.

Aside The Master Theorem: a formal tool for divide-and-conquer

The recursion-tree argument above is one way to solve a recurrence. For the broad class $T(n) = a \cdot T(n/b) + f(n)$ – "solve a subproblems of size n/b, plus do $f(n)$ work to combine them" – there's a closed-form result called the **Master Theorem** (CLRS Ch 4 / OCW r03). Three cases, depending on how $f(n)$ compares to $n^{\log_b a}$ (the work that would happen if you only counted the leaves of the recursion tree):

Case	When	Then $T(n) =$
1. Leaves win	$f(n) = O(n^{\log_b a - \varepsilon})$ for some $\varepsilon > 0$	$\Theta(n^{\log_b a})$
2. Balanced	$f(n) = \Theta(n^{\log_b a})$	$\Theta(n^{\log_b a} \log n)$
3. Root wins	$f(n) = \Omega(n^{\log_b a + \varepsilon})$ + regularity	$\Theta(f(n))$

Merge sort: $T(n) = 2T(n/2) + \Theta(n)$. Here $a = 2, b = 2$, so $\log_b a = 1$, and $f(n) = n = \Theta(n^1)$ – case 2 applies. $T(n) = \Theta(n \log n)$. The hand-derivation above is a worked instance of case 2. The Master Theorem is just the general formula that lets you skip the recursion-tree drawing for any recurrence in this form.

Binary search: $T(n) = T(n/2) + O(1) = 1 \cdot T(n/2) + \Theta(n^0)$, so $\log_b a = 0$, $f(n) = \Theta(n^0)$ – case 2 again. $T(n) = \Theta(\log n)$.

Quicksort (best case): $T(n) = 2T(n/2) + \Theta(n)$, same as merge sort, so $\Theta(n \log n)$. (Worst case requires a different recurrence and falls outside the Master Theorem.)

A handful of useful recurrences and their solutions:

$T(n) = T(n-1) + O(1)$	$\Theta(n)$
$T(n) = T(n-1) + O(n)$	$\Theta(n^2)$
$T(n) = T(n/2) + O(1)$	$\Theta(\log n)$
$T(n) = 2T(n/2) + O(1)$	$\Theta(n)$
$T(n) = 2T(n/2) + O(n)$	$\Theta(n \log n)$
$T(n) = 2T(n/2) + O(n \log n)$	$\Theta(n \log^2 n)$
$T(n) = 2T(n-1) + O(1)$	$\Theta(2^n)$

The full proof of the Master Theorem – including which $f(n)$ shapes fall into which case – is in CLRS Ch 4. For ch_3 the takeaway is that the recursion-tree arguments throughout §3.18 and §3.19 are instances of a general result, not one-off clever tricks.

15.5 Memory cost

Gotcha Merge sort is not in-place

The merge step writes to a temporary buffer of the same size as the range it's merging. At the top of the recursion the buffer is size n. That's $O(n)$ extra memory – dominant over the $O(\log n)$ recursion stack. Quicksort sorts *in place* with only $O(\log n)$ extra memory for the stack; merge sort does not. For huge datasets that don't fit in RAM, or for embedded systems where memory is scarce, this can be the deciding factor.

Aside One-time buffer allocation

A small engineering win: allocate the full-size scratch buffer *once*, outside the recursion, and pass it to each merge call. This avoids $\log n$ distinct allocations. The implementation above reallocates inside `merge` for clarity; production versions pass a buffer.

15.6 Properties

Property	Merge sort
Best case	$\Theta(n \log n)$
Average case	$\Theta(n \log n)$
Worst case	$\Theta(n \log n)$ – guaranteed
Space	$O(n)$ extra – temporary merge buffer
Stable?	Yes
Adaptive?	Not natively (plain merge sort). Variants like Timsort are adaptive.

15.7 Merge sort vs. quicksort

Criterion	Quicksort	Merge sort
Best / avg case	$\Theta(n \log n)$	$\Theta(n \log n)$
Worst case	$\Theta(n^2)$	$\Theta(n \log n)$
Extra memory	$O(\log n)$ (stack)	$O(n)$ (buffer)
Stable?	No	Yes
Cache behavior	Excellent (sequential)	Good (streaming)
In-place?	Yes	No
Recursion depth	$O(\log n)$ avg, $O(n)$ worst	$O(\log n)$ always

Key idea When to reach for each

- **Quicksort** (`std::sort`) when you have plenty of RAM, you don't need stability, and you want typical speed.
- **Merge sort** (`std::stable_sort`) when you need stability or a guaranteed worst case, and you can afford the extra memory.
- **Merge sort on linked lists.** Quicksort's partition assumes random access. Merge sort's *sequential* access pattern makes it the natural choice for linked lists – `std::list::sort` uses it.
- **External sorting (data too large for RAM).** Merge sort is the basis for sorting data on disk: sort chunks that fit in memory, merge them in passes. Quicksort doesn't work that way.

Timsort: merge sort's real-world descendant

Aside Timsort

Python's built-in `sorted` and Java's `Arrays.sort` for objects use **Timsort** – a hybrid merge/insertion sort designed for real-world data. Timsort detects already-sorted runs, sorts small ranges with insertion sort, and merges the runs intelligently. On many real datasets (mostly sorted with pockets of noise) it beats plain quicksort. It's the most commonly-run sorting algorithm in the world today.

Key idea Where this is going

Merge sort completes the picture of general-purpose $O(n \log n)$ sorts: in-place but randomized (quicksort), predictable but memory-hungry (merge sort). Heap sort – coming later with priority queues – is the third option: guaranteed $O(n \log n)$ and in-place, at the cost of being slower in practice and not stable. Each point in the trade-space has a winner.

16 3.20 Radix Sort

Every sort we've seen so far is *comparison-based*: it uses < between pairs of elements as its only decision procedure. The lower bound for that class is $\Omega(n \log n)$. Radix sort is our first non-comparison sort – it never compares two elements to each other – and it breaks the $n \log n$ bound, sorting in $O(n \cdot d)$ where d is the number of digits. For fixed-width integer keys, that's effectively $O(n)$.

16.1 The trick: sort by one digit at a time

Definition — Radix sort

Radix sort sorts an array of integers by processing *one digit at a time*, from the least-significant digit (LSD) to the most-significant. For each digit, it distributes values into 10 **buckets** (one per digit value $0, 1, \ldots, 9$), then rebuilds the array by concatenating the buckets in order. After processing all digits, the array is sorted.

Key idea Why this works: stability of the digit pass

After the first pass, the array is sorted by the 1's digit. After the second pass – *sorted by the 10's digit in a stable way* – ties on the 10's digit keep the order established by the first pass, so the array is now sorted by "10's digit, then 1's digit" – which is numerical order up to two digits. Repeating for higher digits extends the sorted-ness to the full number. Radix sort is correct *only because* the digit-by-digit pass is stable.

16.2 The LSD algorithm

Example Radix sort in C++, non-negative integers

```
void radixSort(std::vector<int>& nums) {
    if (nums.empty()) return;

    int maxVal = *std::max_element(nums.begin(), nums.end());
    std::vector<std::vector<int>> buckets(10);

    for (int pow10 = 1; maxVal / pow10 > 0; pow10 *= 10) {
        for (int v : nums) {
            int digit = (v / pow10) % 10;
            buckets[digit].push_back(v);
        }
        std::size_t idx = 0;
        for (auto& bucket : buckets) {
            for (int v : bucket) nums[idx++] = v;
```

```
            bucket.clear();
        }
    }
}
```

Outer loop advances through the digits (ones, tens, hundreds, …). Inner first loop distributes values to buckets; inner second loop gathers them back in order. The pass is stable because we iterate `nums` in order and `push_back` preserves arrival order within a bucket.

Gotcha Stability of the gather step is essential

If you reorder within a bucket – for example, sorting or randomizing each bucket's contents – radix sort breaks. The whole algorithm depends on each digit pass being a *stable* partial sort.

Tracing a small example

Step	Array
Input	170, 45, 75, 90, 802, 24, 2, 66
After 1's pass	170, 90, 802, 2, 24, 45, 75, 66
After 10's pass	802, 2, 24, 45, 66, 170, 75, 90
After 100's pass	2, 24, 45, 66, 75, 90, 170, 802

Three passes, because the max value has three digits. After each pass, the prefix of digit-sorted-ness grows by one.

16.3 Runtime

Key idea $O(n \cdot d)$

Let n be the number of elements and d be the number of digits of the maximum value. Each pass does $O(n)$ work (distribute + gather). There are d passes. Total: $\Theta(n \cdot d)$.
For 32-bit integers, $d \leq 10$. For 64-bit integers, $d \leq 20$. Those are *constants* – independent of n. So for fixed-width integers, radix sort is effectively $\Theta(n)$.

Gotcha "Beats $n \log n$" comes with fine print

Yes, radix sort is linear in n when keys are fixed-size integers. But:

- The constant factor hides "$\times d$," which can be larger than $\log_2 n$ for moderate n. Radix sort usually beats quicksort only for large n with narrow keys.
- It doesn't generalize: radix sort only works on things you can decompose into digits (integers, fixed-length strings, fixed-precision floats with bit manipulation).
- It's not in-place in the naïve version: 10 buckets of potentially n elements each, plus copy-back, is $O(n)$ extra memory.

It's a specialist tool, not a `std::sort` replacement.

16.4 Handling negatives and other bases

Example Signed integers: post-process into two buckets

The simple LSD algorithm above sorts by *absolute value* when negatives are present (since `-12` has 1's digit 2). The textbook's fix: after the main radix sort, split into a `negatives` bucket and a `nonNegatives` bucket, *reverse* the negatives (so $-42, -12, -78$ becomes $-78, -42, -12$), and concatenate `[reversed negatives] ++ [nonNegatives]`.

Aside Radix = base

The number of buckets you use is the *radix* or base. Ten is natural for decimal integers, but you can use any base. **Base 256** (one byte at a time) is the usual choice for radix-sorting binary integer keys on modern computers – one memory read per digit, exactly 4 passes for a 32-bit int. Base 2 (one bit at a time) takes more passes but uses only 2 buckets. Picking the base is a time / memory trade-off.

16.5 Properties

Property	Radix sort (LSD)
Time (best / avg / worst)	$\Theta(n \cdot d)$ – all cases the same
Space	$\Theta(n + B)$ where B is the base
Stable?	Yes – and correctness depends on it
Adaptive?	No – doesn't notice already-sorted input
Works on?	Integer keys (or anything decomposable into fixed-width digits)

16.6 When to reach for radix sort

Key idea Niche but powerful

- **Huge volumes of integers or fixed-length strings.** Log files with numeric IDs, network packet sorting by IP address, database sorts on integer keys – radix sort can be 3–10× faster than quicksort.
- **Hardware where comparison is expensive.** GPUs and FPGAs often use radix sort because distribution-style algorithms parallelize cleanly.
- **Fixed-width keys with a small base.** Sort 8-bit values with a single 256-bucket pass – basically free.

For variable-length strings, general objects, or anything that can only be compared (not decomposed), stick to `std::sort`.

16.7 Counting sort: the standalone non-comparison sort

Radix sort uses counting sort once per digit. But counting sort is also a complete sorting algorithm in its own right – the right choice when keys are integers in a known small range and you want $O(n + k)$ time, where k is the size of the range.

Definition — Counting sort

Counting sort sorts integer keys in the range $[0, k)$ by: (1) counting how many times each key value occurs, (2) computing cumulative sums to determine each value's final output position, and (3) walking the input in reverse to place each element at its position (decrementing the count as we go). $\Theta(n + k)$

time, $\Theta(n+k)$ space, stable.

Example Counting sort in C++

```
void countingSort(std::vector<int>& nums, int k) {
    // Assumes every nums[i] satisfies 0 <= nums[i] < k.
    std::vector<int> count(k, 0);
    for (int v : nums)              ++count[v];          // count
        occurrences
    for (int i = 1; i < k; ++i)     count[i] += count[i-1]; // cumulative
    std::vector<int> out(nums.size());
    for (int i = nums.size() - 1; i >= 0; --i) {         // walk BACKWARD
        out[--count[nums[i]]] = nums[i];
    }
    nums = std::move(out);
}
```

The cumulative-sum array tells you exactly where each value's "block" ends in the output. Walking the input *backward* and decrementing as you go preserves the original order of equal keys – that's why counting sort is stable.

Gotcha Why backward, why decrement?

If you walked the input forward and *incremented*, equal keys would land in their final block in *reverse* input order – breaking stability. The walk-backward + decrement convention is what makes counting sort a stable building block, which is exactly the property radix sort needs.

Key idea When counting sort wins

- **Grades 0–100.** $k = 101$ buckets, n students. Sort by score in $\Theta(n + 101) = \Theta(n)$.
- **ASCII characters.** $k = 128$. Sort a string by character in $\Theta(n)$.
- **Pixel intensities, age in years, days of the year.** Anywhere keys live in a known small range.

Counting sort breaks down when k is huge – $k = 10^9$ means a billion-entry array even if $n = 10$. That's where radix sort re-enters: chop the key into base-n digits and run counting sort on each digit, getting $\Theta(n)$ for fixed-width integer keys.

Key idea Where this is going

Radix sort closes out the sorting tour. You now have the comparison-based trio – insertion, quicksort, merge sort – plus the specialized non-comparison contender, radix sort. The Big-O machinery you used throughout this chapter (formalized in §3.10) is the cost-analysis foundation for every later chapter. Chapter 4 starts the per-structure walk – linked lists, the alternative to contiguous storage – and every performance comparison there is phrased in the vocabulary you built here.

17 3.21 Overview of Fast Sorting Algorithms

The last several sections introduced a parade of sorting algorithms. This section steps back and classifies them. Two questions organize the view: **how fast?** (average-case complexity) and **what kind?** (does the

algorithm compare elements, or does it exploit more structure in the keys?).

Definition — Fast sorting algorithm

A **fast sorting algorithm** is one whose average-case runtime is $O(n \log n)$ or better. Sorts that average $\Theta(n^2)$ – or anything slower than $n \log n$ – are considered "slow" regardless of their constant factor.

17.1 The speed classification

Algorithm	Average case	Fast?
Selection sort	$\Theta(n^2)$	No
Insertion sort	$\Theta(n^2)$	No
Shell sort	$\Theta(n^{1.5})$ (typical sequences)	No
Quicksort	$\Theta(n \log n)$	Yes
Merge sort	$\Theta(n \log n)$	Yes
Heap sort	$\Theta(n \log n)$	Yes
Radix sort	$\Theta(n \cdot d) = O(n)$ for fixed-width	Yes

Key idea The $n \log n$ frontier

Three comparison sorts meet the $n \log n$ bar on average: **quicksort, merge sort, heap sort**. They're not interchangeable – they differ on worst case, memory, stability, and cache behavior – but they're the short list for "fast comparison sort." Shell sort sits just below, in the $n^{1.5}$ band; insertion and selection sort sit well below, in the n^2 band.

17.2 The $\Omega(n \log n)$ lower bound for comparison sorts

Key idea Why no comparison sort can do better than $n \log n$

A sorting algorithm that uses only pairwise comparisons is deciding which of $n!$ possible input permutations it's looking at. Each comparison has two outcomes, so after k comparisons the algorithm has distinguished among at most 2^k permutations. For the algorithm to correctly identify any of $n!$ inputs, we need

$$2^k \geq n!, \quad \text{so} \quad k \geq \log_2(n!) \in \Theta(n \log n).$$

This is a *lower bound on the problem*, not on any particular algorithm. Quicksort, merge sort, and heap sort all meet it (up to constants).

17.3 Comparison vs. non-comparison sorts

Definition — Comparison sort

A **comparison sort** uses only the operation "compare two elements" to order them. Every sort except radix sort we've seen is comparison-based.

Algorithm	Comparison sort?
Selection, insertion, shell sort	Yes
Quicksort, merge sort, heap sort	Yes
Radix sort (and counting, bucket sort)	No

Key idea Why radix sort can beat the lower bound

The $\Omega(n \log n)$ lower bound applies only to comparison sorts. Radix sort doesn't compare elements; it looks at digits directly, which is additional structural information. The lower bound doesn't apply, and radix sort achieves $\Theta(n)$ on fixed-width integer keys.

The trade-off is universality. A comparison sort works on anything you can define < for – strings, structs, floats, custom types. Radix sort needs keys you can decompose into digits. This is why `std::sort` is comparison-based even though radix sort could be faster for certain inputs: the library has to work on *any* `operator<`-able type.

Aside Bucket sort: the third non-comparison sort

The third member of the non-comparison family is **bucket sort**: assume input values are uniformly distributed in $[0, 1)$, divide the range into n equal-width buckets, drop each input into its bucket, sort each bucket with insertion sort, and concatenate. $\Theta(n)$ expected time on uniform input, $\Theta(n^2)$ worst case. Useful for floating-point keys or other values you can predict the distribution of. Full coverage in §13.3 of the extras chapter.

17.4 Best / average / worst, side by side

Average case is the headline number, but the other two cases matter when you care about guarantees (worst) or optimized input (best).

Algorithm	Best	Average	Worst
Quicksort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$
Merge sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Heap sort	$\Theta(n)$ * / $\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Radix sort	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$

Aside The $\Theta(n)$ heap-sort best case

The textbook lists heap sort's best case as $\Theta(n)$. That number refers to the cost of *building* the heap, which is $\Theta(n)$ by a clever analysis. The subsequent extraction phase is $\Theta(n \log n)$, so total heap sort work is still $\Theta(n \log n)$ – the "$\Theta(n)$ best case" applies only to the heapify stage, not the whole sort. Heap sort the algorithm is $\Theta(n \log n)$ in all cases.

17.5 Picking a fast sort: a mini decision tree

Key idea The 80/20 rule

1. **Just use `std::sort`.** Introsort. Handles almost every case well.
2. **Need stability?** `std::stable_sort` (merge sort). Slightly slower; preserves order of equal keys.
3. **Can't afford worst-case blowup?** Merge sort or heap sort. Both give $O(n \log n)$ worst case; merge sort uses $O(n)$ extra memory, heap sort is in-place.
4. **Sorting integers with a narrow range, huge n?** Radix sort – the one place $O(n)$ is actually achievable.
5. **Data mostly sorted?** Timsort (merge sort variant) or insertion sort for tiny n. Both exploit existing order.

6. **Small n (< 32ish)?** Insertion sort wins on constants. This is why `std::sort` falls back to insertion sort at the leaves.

17.6 The extended comparison table

Everything from the chapter, in one place:

Algorithm	Best	Average	Worst	Space	Stable?	Adaptive?
Selection	n^2	n^2	n^2	1	No	No
Insertion	n	n^2	n^2	1	Yes	Yes
Shell	$n \log n$	$n^{1.5}$	$n^{1.5\text{–}2}$	1	No	Some
Quicksort	$n \log n$	$n \log n$	n^2	$\log n$	No	No
Merge	$n \log n$	$n \log n$	$n \log n$	n	Yes	No
Heap	$n \log n$	$n \log n$	$n \log n$	1	No	No
Radix	n	n	n	n	Yes	No

Gotcha The "O" notation hides constants

Equal Big-O doesn't mean equal speed. On random data, quicksort's inner loop is two or three times faster than merge sort's – because quicksort's partition is extremely cache-friendly and writes only when it has to. "$n \log n$" is the right first-order answer, but the second-order constants decide which $n \log n$ algorithm actually wins on your hardware with your data.

Key idea Where this is going

This closes the sorting tour. The tools you built here – Big-O, the growth hierarchy, and a feel for comparison vs. non-comparison sorts – are load-bearing for every remaining chapter. Linked-list search in ch. 4 is linear time (and you'll know why without thinking). BST search in ch. 6 is $O(\log n)$ amortized but $O(n)$ worst-case (and you'll know which shape of tree triggers which). Hash table lookup in ch. 5 is $O(1)$ expected (and you'll know what "expected" is buying you).

What this connects to

Aside Forward links to later chapters

- **Chapter 4 (linked lists).** Every performance comparison in ch. 4 is "compared to a vector, …". The complexity vocabulary from 3.9 and 3.10 is what lets those comparisons land. Linked-list traversal is $O(n)$ because each step is constant and there are n of them – the exact reasoning you practiced here.
- **Chapter 5 (hash tables).** The big claim – "hash lookup is $O(1)$ expected" – only makes sense if you're fluent with expected vs. worst case, which is a 3.10 skill. The chaining and probing strategies there are extensions of the collision idea introduced at the end of 3.1.
- **Chapter 6 (BSTs).** BST insert/search/remove are $O(h)$, which is $O(\log n)$ on a balanced tree and $O(n)$ on a degenerate one. The "balanced" piece is literally about whether the tree shape keeps you on the log curve from 3.10's growth hierarchy.
- **Sorting choices in later work.** Whenever you see `std::sort` (introspection sort, essentially quicksort + heapsort fallback) you're looking at the quicksort analysis from 3.18. When you see

`std::stable_sort`, that's merge sort from 3.19 with its stability guarantee.

Companion materials. One-page cheat sheet: `cheat_sheets/ch_3.tex` (includes the full sort comparison table). Practice prompts (problem → pseudocode → C++): `practice_prompts/ch_3.md`.